

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
УКРАЇНИ «КПІ ІМ. ІГОРЯ СІКОРСЬКОГО»**

ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ

**Кафедра системного програмування і спеціалізованих
комп'ютерних систем**

КУРСОВА РОБОТА

з дисципліни "Структури даних і алгоритми"

Виконала: Поліщук М. І.

Група: КВ-33

Номер залікової книжки:

Допущений до захисту

2 семестр 2023/2024 навчального року

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
УКРАЇНИ «КПІ ІМ. ІГОРЯ СІКОРСЬКОГО»**

ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ

**Кафедра системного програмування і спеціалізованих
комп'ютерних систем**

Узгоджено

ЗАХИЩЕНА "___" _____ 2024р.

Керівник роботи

з оцінкою _____

_____/Марченко О.І./

_____/Марченко О.І./

***Дослідження ефективності методів
сортування (Алгоритм сортування №6 методу прямого
вибору, Гібридний алгоритм "вставка – обмін",
Гібридний алгоритм "вибір №3 – обмін") на
багатовимірних масивах***

Виконавець роботи: _____



(підпис)

Поліщук Марія Ігорівна

_____ 2024р.

ТЕХНІЧНЕ ЗАВДАННЯ НА КУРСОВУ РОБОТУ

I. Описати теоретичні положення, від яких відштовхується дослідження, тобто принцип та схему роботи кожного із досліджуваних алгоритмів сортування для одновимірного масива, навести загальновідомі властивості цих алгоритмів та оцінки кількості операцій порівняння та присвоєння для них.

II. Скласти алгоритми рішення задачі сортування в багатовимірному масиві заданими за варіантом методами та написати на мові програмування за цими алгоритмами програму, яка відповідає вимогам розділу «Вимоги до програми курсової роботи».

III. Виконати налагодження та тестування коректності роботи написаної програми.

IV. Провести практичні дослідження швидкодії складених алгоритмів, тобто виміри часу роботи цих алгоритмів для різних випадків та геометричних розмірів багатовимірних масивів.

V. За результатами досліджень скласти порівняльні таблиці за різними ознаками.

- Одна таблиця результатів (вимірів часу сортування впорядкованого, випадкового і обернено-впорядкованого масива)

для масива з заданими геометричними розмірами повинна бути такою:

Таблиця № для масива $A[P,M,N]$, де $P=$; $M=$; $N=$;

	Впорядко- ваний	Невпорядко- ваний	Обернено впо- рядкований
Назва алгоритму 1			
Назва алгоритму 2			
Назва алгоритму 3			

- Для варіантів курсової роботи, де крім алгоритмів порівнюються також способи обходу, в назвах рядків таблиць потрібно вказати як назви алгоритмів, так і номери способів обходу.
- Для виконання ґрунтовного аналізу алгоритмів потрібно зробити виміри часу та побудувати таблиці для декількох масивів з різними геометричними розмірами.
- Зробити виміри часу для стандартного випадку одномірного масива, довжина якого вибирається такою, щоб можна було виконати коректний порівняльний аналіз з рішенням цієї ж задачі для багатовимірного масива.
- Кількість необхідних таблиць для масивів з різними геометричними розмірами залежить від задачі конкретного варіанту курсової роботи і вибираються так,

щоб виконати всебічний та ґрунтовний порівняльний аналіз заданих алгоритмів.

- Рекомендації випадків дослідження з різними геометричними розмірами масивів наведені у розділі «Випадки дослідження».

VI. Для наочності подання інформації за отриманими результатами рекомендується також будувати стовпчикові діаграми та графіки.

VII. Виконати порівняльний аналіз поведінки заданих алгоритмів за отриманими результатами (вимірами часу):

- для одномірного масива відносно загальновідомої теорії;
- для багатовимірних масивів відносно результатів для одномірного масива;
- для заданих алгоритмів на багатовимірних масивах між собою;
- дослідити вплив різних геометричних розмірів багатовимірних масивів на поведінку алгоритмів та їх взаємовідношення між собою;
- **для всіх вищезазначених пунктів порівняльного аналізу пояснити, ЧОМУ алгоритми в розглянутих ситуаціях поведуть себе саме так, а не інакше.**

VIII. Зробити висновки за зробленим порівняльним аналізом.

IX. Програму курсової роботи під час її захисту **ОБОВ'ЯЗКОВО** мати при собі на електронному носії інформації.

ВАРІАНТ № 156

Варіант	Задача	Номери алгоритмів згідно нумерації у розділі «Алгоритми сортування» (в дужках – окремий спосіб обходу)	Способи обходу
156	1	10, 17, 20	3

ЗАДАЧА

1. Впорядкувати окремо кожен переріз тривимірного масива $Arr3D$ $[P, M, N]$ наскрізно по стовпчиках за незменшенням.

ДОСЛІДЖУВАНІ МЕТОДИ ТА АЛГОРИТМИ

10. Алгоритм сортування №6 методу прямого вибору (додаток 1, рис. №10).
17. Гібридний алгоритм "вставка – обмін" (додаток 1, рис. №17).
20. Гібридний алгоритм "вибір №3 – обмін" (додаток 1, рис. №20).

СПОСОБИ ОБХОДУ

3. Виконати сортування, здійснюючи обхід безпосередньо по елементах заданого двовимірного масива, не використовуючи додаткових масивів і перетворень індексів.

ВИПАДКИ ДОСЛІДЖЕННЯ

Випадок дослідження I. Залежність часу роботи алгоритмів від форми перерізу масива.

Рекомендовані розміри масива для досліджень:

$P = \text{const} = 3$, $M = \text{var}$, $N = \text{var}$, $M*N = \text{const} = 100000000$.

1) $M = 10$; $N = 10000000$;

1) $M = 100$; $N = 1000000$;

2) $M = 1000$; $N = 100000$;

3) $M = 10000$; $N = 10000$;

4) $M = 100000$; $N = 1000$;

5) $M = 1000000$; $N = 100$;

6) $M = 10000000$; $N = 10$;

Вектор довжиною $NV = M*N = 100000000$.

Для порівняння час сортування вектора довжиною $NV = M*N$ помножити на P .

Випадок дослідження II. Залежність часу роботи алгоритмів від кількості перерізів масива

Рекомендовані розміри масива для досліджень:

$P = \text{var}$, $M = \text{const} = 2000$, $N = \text{const} = 2000$. (кількість елементів у перерізі $M*N = 4000000$)

1) $P = 2$;

2) $P = 4$;

3) $P = 8$;

4) $P = 16$;

5) $P = 32$;

6) $P = 64$;

Час сортування вектора довжиною $NV = 4000000$ (дорівнює кількості елементів у перерізі) помножити на P . Вимір і порівняння для вектора достатньо зробити тільки для найбільшого P .

Випадок дослідження III. Залежність часу роботи алгоритмів від розміру перерізів масива

Порівняння з вектором для цього випадку не обов'язкове.

Для бажаючих порівняти з вектором: тривимірний масив кожного розміру $P \times M \times N$ (де $M=N$) потрібно окремо порівнювати з вектором відповідної довжини $M \times N$, і час сортування такого вектора помножити на кількість перерізів P .

Рекомендовані розміри масива для досліджень:

$P = \text{const} = 3$

- 1) $M = N = 4$ ($M \times N = 16$)
- 2) $M = N = 8$ ($M \times N = 64$)
- 3) $M = N = 16$ ($M \times N = 256$)
- 4) $M = N = 32$ ($M \times N = 1024$)
- 5) $M = N = 64$ ($M \times N = 4096$)
- 6) $M = N = 128$ ($M \times N = 16384$)
- 7) $M = N = 256$ ($M \times N = 65536$)
- 8) $M = N = 512$ ($M \times N = 262144$)
- 9) $M = N = 1024$ ($M \times N = 1048576$)
- 10) $M = N = 2048$ ($M \times N = 4194304$)
- 11) $M = N = 4096$ ($M \times N = 16777216$)
- 12) $M = N = 8192$ ($M \times N = 67108864$)

ОПИС ТЕОРЕТИЧНИХ ПОЛОЖЕНЬ

1.Алгоритм сортування №6 методу прямого вибору

Принцип та загальна схема сортування

В кожний момент часу масив вважається поділеним на 2 частини:

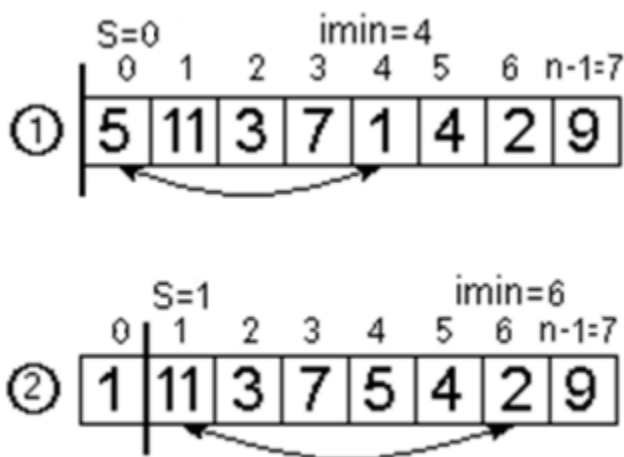
- 1) **вже відсортовану** частину;
- 2) **ще невідсортовану** частину.

Перед початком сортування весь масив вважається невідсортованою частиною.

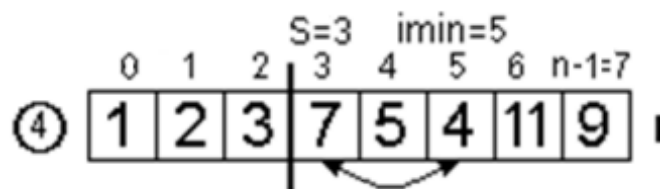
Загальний принцип сортування №6 методом прямого вибору:

1. Вибираємо в масиві найменший елемент в діапазоні індексів від **0** до **n-1**.
2. Знайдений мінімальний елемент міняємо місцями з нульовим елементом масиву тільки ,якщо позиції цих двох елементів не збігаються. Таким чином один елемент вже є відсортованим.
3. Дії пунктів 1-2 повторюються для ще невідсортованої частини в діапазоні від **1** до **n-1**, від **2** до **n-1** і т.д., міняючи черговий найменший елемент , за умовою що він не є початковим взятим за мінімальний, місцями з елементами відповідно з індексом **1**, з індексом **2** і т.д. Таким чином відсортована частина зростає.
Останній доцільний індексний діапазон пошуку мінімуму – від **n-2** до **n-1**.

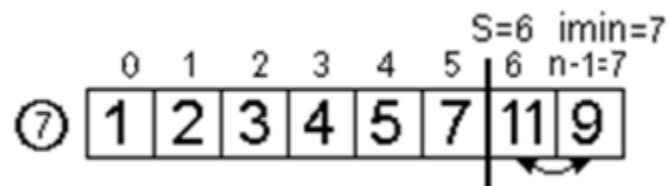
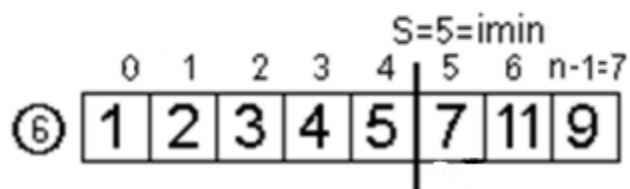
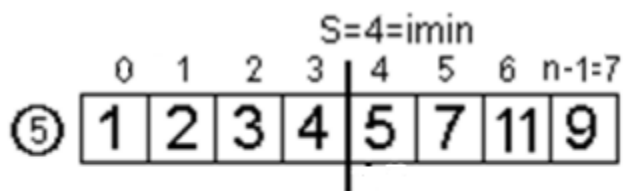
Загальна схема проходів сортування:



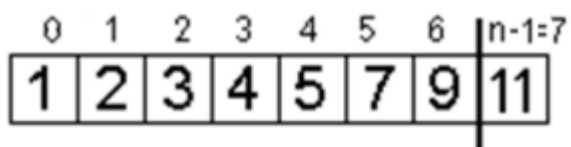
На третьому кроці елементи не міняються місцями, оскільки не виконується умова ($S \neq imin$).



На п'ятому та шостому кроці елементи не міняються місцями, оскільки не виконується умова ($S \neq imin$).



Результат:



Для реалізації використовуються наступні змінні:

n – довжина масиву **A**;

A – масив, що сортується;

s - початковий (стартовий) індекс позиції, з якої починається черговий прохід з пошуком мінімального елемента (знайдений мінімальний елемент буде переставлений саме на цю позицію, якщо він там вже не стоїть);

imin - індекс поточного мінімального елемента;

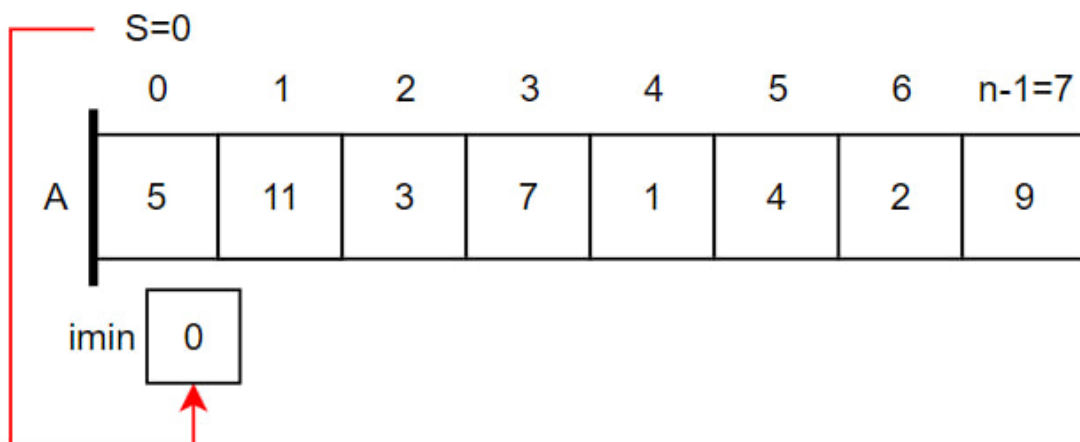
i - індекс поточного елемента масиву **A**, взятого для порівняння;

tmp - тимчасова комірка для обміну місцями двох елементів масиву.

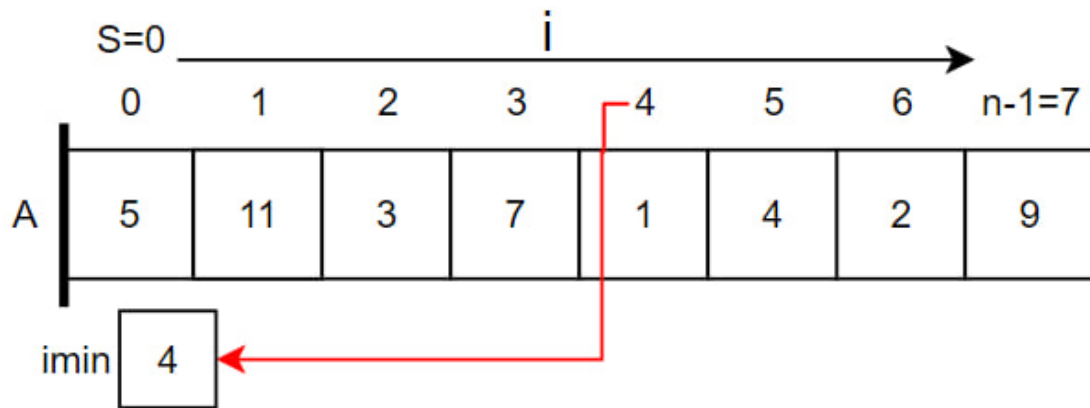
Схема одного проходу Алгоритму сортування №6 методу прямого вибору

Крок 1. Початкові установки змінних $imin = s$;

Змінна **S** є лічильником циклу по проходах.



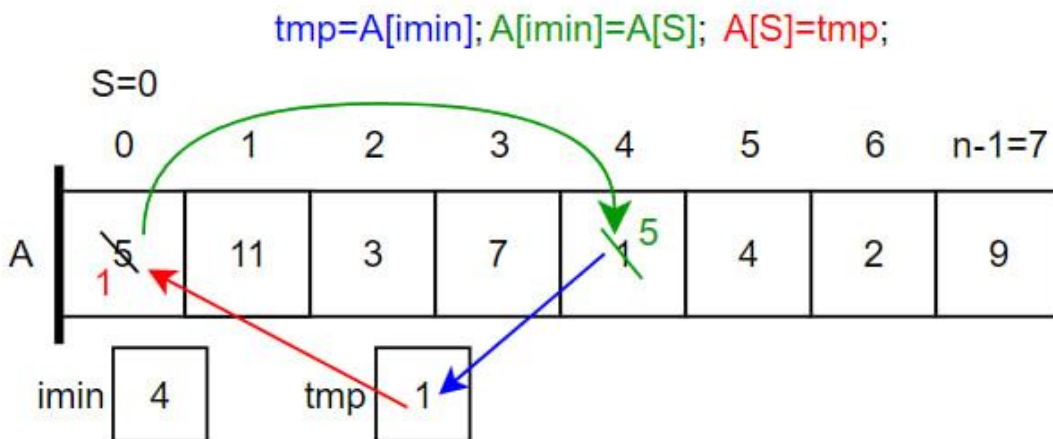
Крок 2. Пошук мінімального елемента масиву. Результат буде таким.



Крок 3. Перевірка чи не співпадає індекс мінімального елемента з індексом лічильника S.

(imin!=s)
imin 4 != S=0

Крок 4. Після перевірки треба виконати всі три присвоєння обміну місцями мінімального та S-го елементів:



Алгоритм сортування №6 методу прямого вибору на мові C:

```
clock_t Select6(int *A, int N)
{
```

```

int imin, tmp;
clock_t time_start, time_stop;
time_start = clock();
for(int s=0; s<N-1; s++){
    imin=s;
    for(int i=s+1; i<N; i++)
        if (A[i]<A[imin]) imin=i;
    if (imin!=s) {
        tmp=A[imin];
        A[imin]=A[s];
        A[s]=tmp;
    }
}
time_stop = clock();
return time_stop - time_start;
}

```

2. Гібридний алгоритм "вставка – обмін"

Принцип та загальна схема сортування

Цей алгоритм є поєднанням двох відомих алгоритмів. Сортування вставкою та сортування обміном. За основу береться Алгоритм 2 вставки, в поєднанні з обміном.

В кожний момент часу масив вважається поділеним на 2 частини:

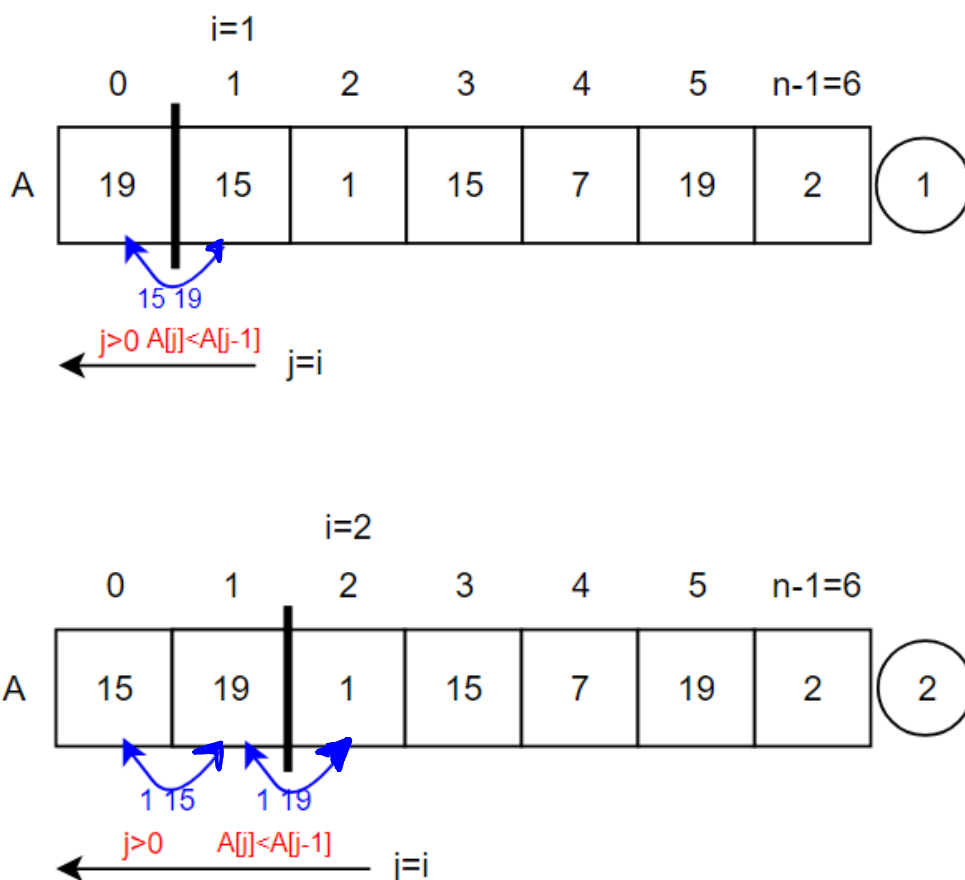
- 1) **вже відсортовану** частину;
- 2) **ще невідсортовану** частину.

Перед початком сортування за відсортовану частину приймається 1-й елемент масиву (з індексом 0).

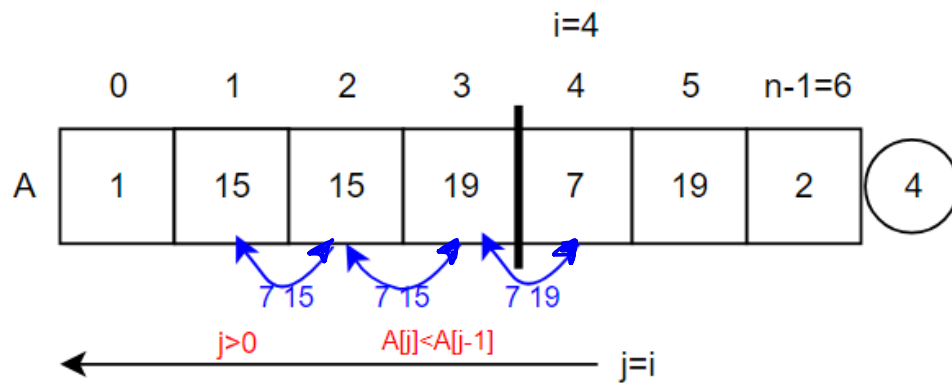
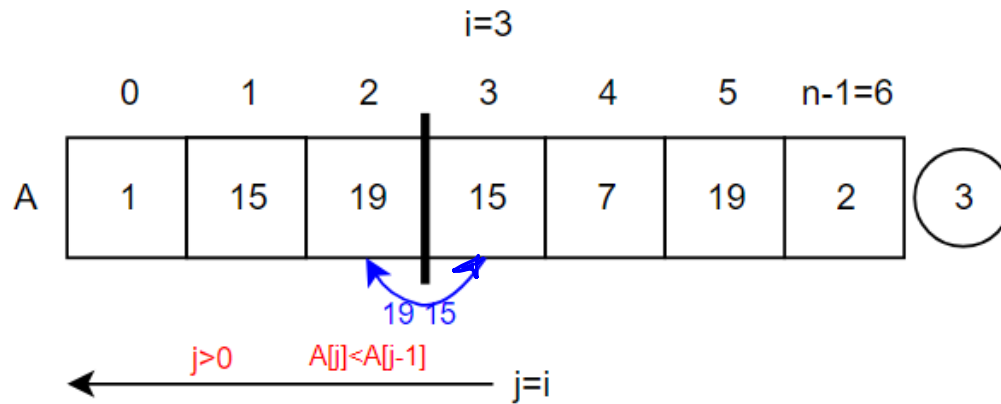
Загальний принцип сортування методом “вставка – обмін”:

1. Беремо черговий елемент з невідсортованої частини.
2. Проходимося від цього елемента до початку масиву поки виконується умова ($j > 0 \ \&\& \ A[j] < A[j-1]$).
3. Паралельно з пунктом 2, поки виконується умова ($j > 0 \ \&\& \ A[j] < A[j-1]$) обмінюємо між собою сусідні елементи, як в алгоритмі обміну.
4. Пункти 1-3 повторюються для всіх елементів невідсортованої частини.

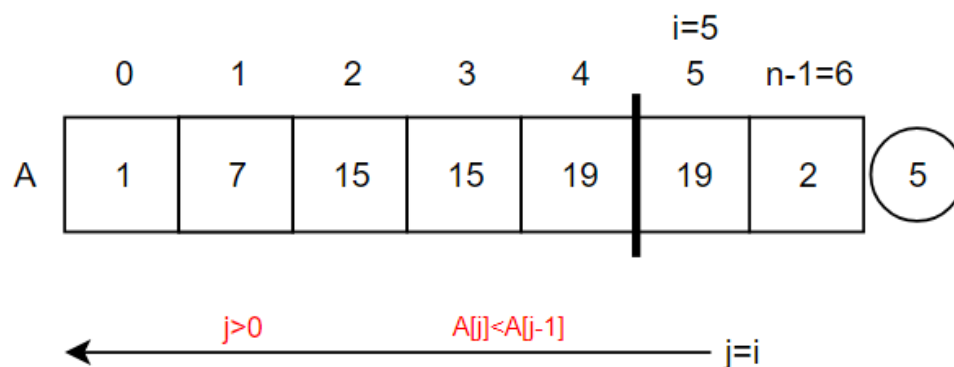
Загальна схема проходів сортування:

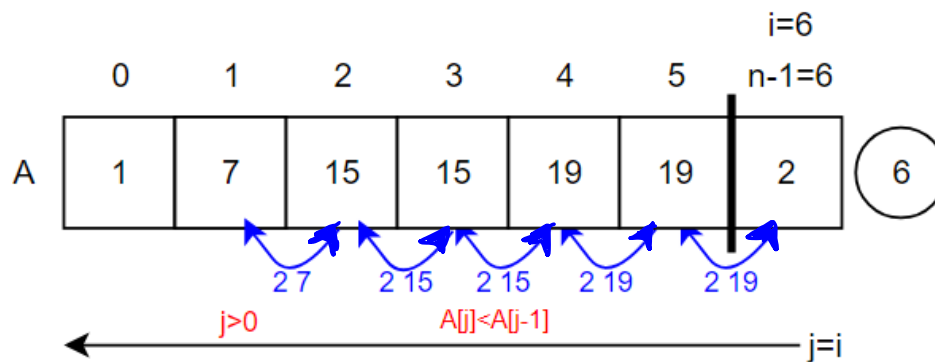


На третьому кроці елементи, на позиції після 2 індексу, не міняються місцями оскільки умова продовження циклу ($j > 0 \ \&\& \ A[j] < A[j-1]$) не виконується.

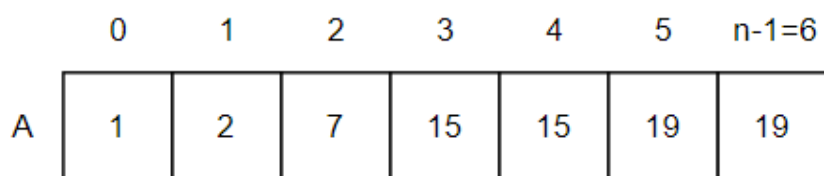


На п'ятому кроці елементи не міняються місцями оскільки умова початку циклу ($j>0 \ \&\& \ A[j] < A[j-1]$) не виконується.





Результат:



Для реалізації використовуються наступні змінні:

n – довжина (кількість елементів) масиву **A**;

A – масив, що сортується;

i – індекс поточного елемента масиву **A** з якого починається порівняння, значення для передачі в індекс **j**;

j – індекс поточного елемента масиву **A**, взятого для порівняння **j**-го та (**j-1**)-го елементів;

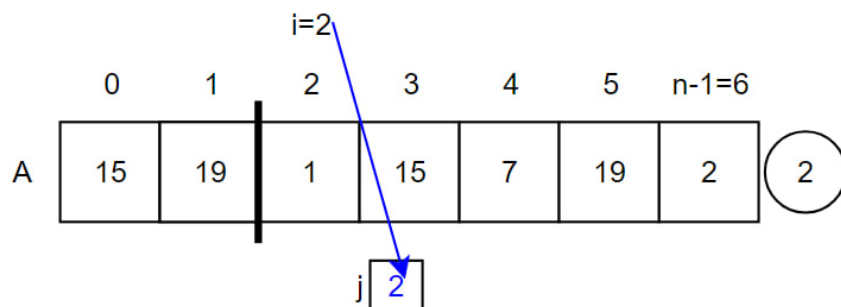
tmp – тимчасова комірка для обміну місцями двох елементів масиву.

Схема одного проходу гібридного алгоритму "вставка – обмін"

Демонстрація роботи алгоритму з другого кроку, для наочності.

Крок 1. Початкові установки змінних **j = i**;

Змінна **i** є лічильником циклу по проходах.



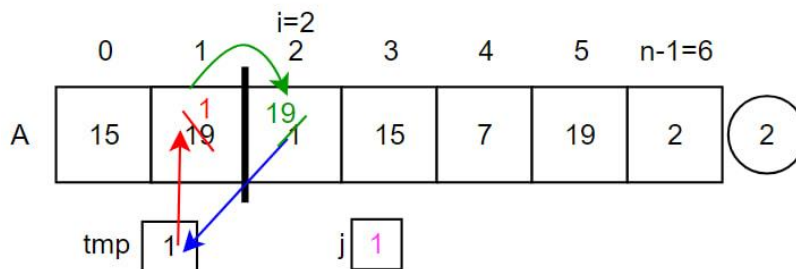
Крок 2. Перевірка виконання умови

$(j > 0 \ \&\& \ A[j] < A[j-1])$

j 2 $A[j] =$ 1 $< A[j-1] =$ 19

Крок 3. Умова виконується, тому обмінюємо сусідні елементи з індексами j та $j-1$ місцями. Зменшуємо індекс j на одиницю.

$tmp = A[j]; A[j] = A[j-1]; A[j-1] = tmp; j = j - 1;$



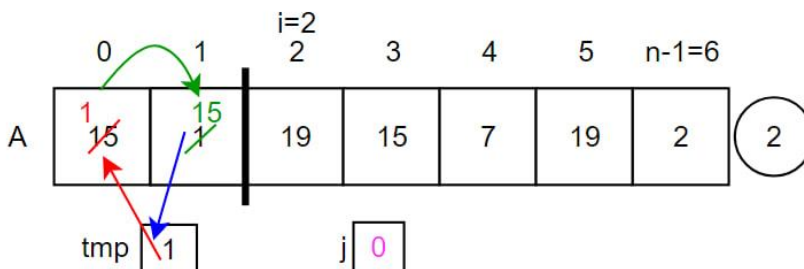
Крок 4. Після виконання тіла умовної конструкції, повторно перевіряємо умову продовження циклу.

$(j > 0 \ \&\& \ A[j] < A[j-1])$

j 1 $A[j]$ 1 $< A[j-1] =$ 15

Крок 5. Умова виконується, тому повторюємо дії **кроку 3** для елементів з новими індексами.

$tmp = A[j]; A[j] = A[j-1]; A[j-1] = tmp; j = j - 1;$



Крок 6. Виконуємо перевірку як в **кроках 2** та **4**.

$(j > 0 \ \&\& \ A[j] < A[j-1])$

j 0

Умова не виконується, тому переходимо на наступну ітерацію **i**;

Гібридний алгоритм "вставка – обмін" на мові C:

```
clock_t InsertExchange(int *A, int N)
```

```
{
```

```
    int j, tmp;
```

```
    clock_t time_start, time_stop;
```

```
    time_start = clock();
```

```
    for(int i=1; i<N; i++){
```

```
        j=i;
```

```
        while (j>0 && A[j]<A[j-1]) {
```

```
            tmp=A[j];
```

```
            A[j]=A[j-1];
```

```
            A[j-1]=tmp;
```

```
            j=j-1;
```

```
        }
```

```
    }
```

```
    time_stop = clock();
```

```
    return time_stop - time_start;
```

```
}
```

3. Гібридний алгоритм "вибір№3 – обмін"

Принцип та загальна схема сортування

Цей алгоритм є поєднанням двох відомих алгоритмів. Сортування вставкою та сортування обміном. Цей алгоритм має трохи спільного з Алгоритмом 4 сортування прямим методом обміну (Шейкерне сортування). Він також має ліву та праву границі які рухаються на зустріч, допоки виконується умова ($L < R$).

В кожний момент часу масив вважається поділеним на 3 частини:

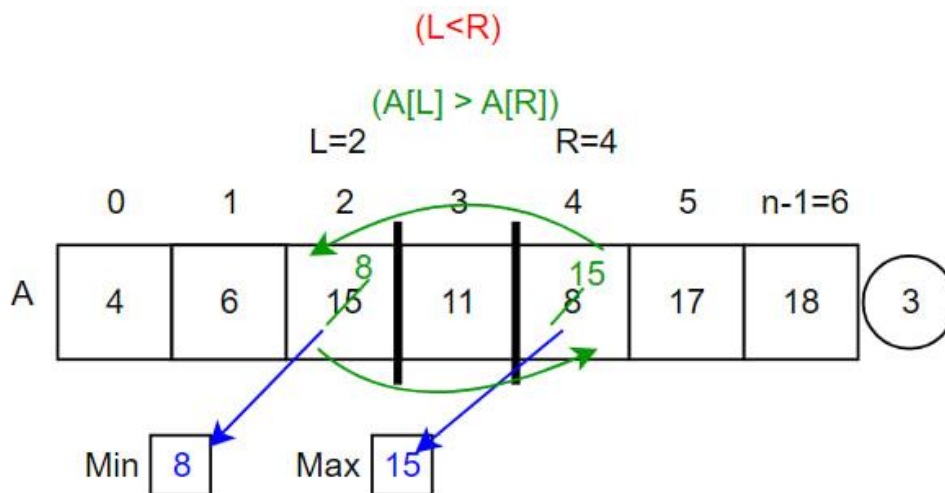
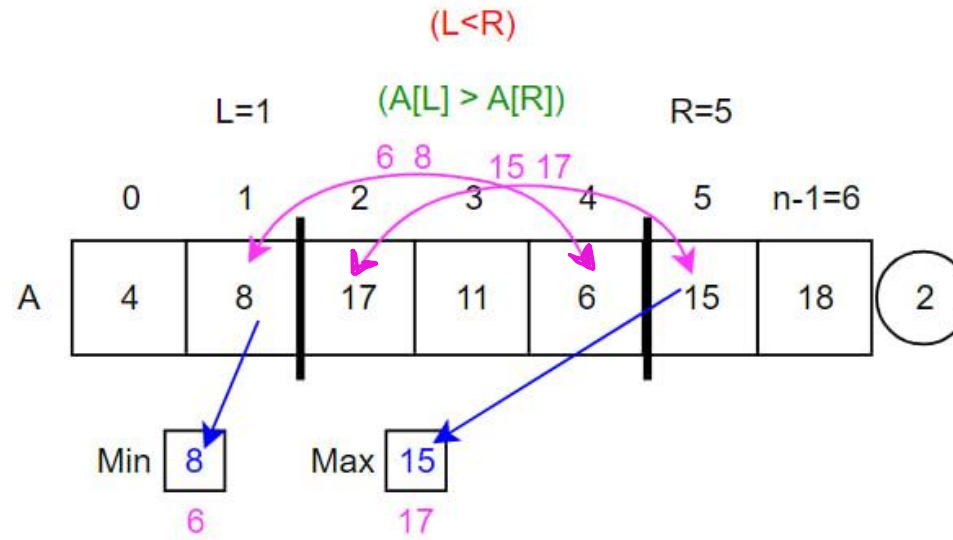
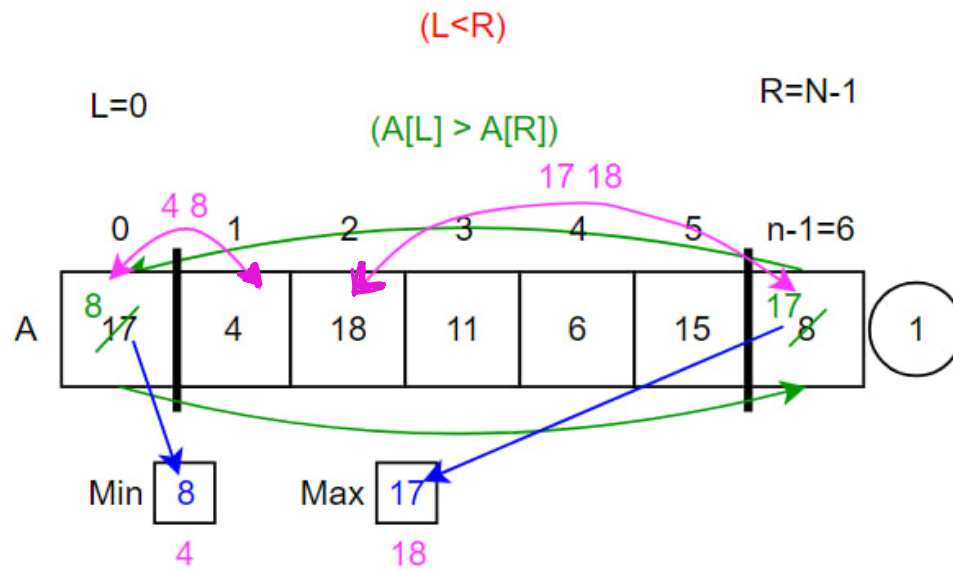
- 1) **вже відсортована з початку масиву;**
- 2) **вже відсортована в кінці масиву;**
- 3) **ще невідсортована частина між ними;**

Перед початком сортування за відсортовану частину приймається 1-й та останні елементи масиву (з індексом 0 та $n-1$). Які одразу порівнюються між собою на предмет більшого та меншого.

Загальний принцип сортування гібридним алгоритмом "вибір№3 – обмін"

1. При першому проході, беремо ліву та праву границі **L** і **R**, та присвоюємо їм перший та останній елемент відповідно. При подальших проходах, ліва та права границі беруться з індексами на одиницю більша та на одиницю менша відповідно.
2. Порівнюємо **A[L]** та **A[R]** між собою. Менший елемент ставимо на позицію з індексом **L** та присвоюємо в змінну **Min**, більший елемент ставимо на позицію з індексом **R** та присвоюємо в змінну **Max**.
3. Проходимося по елементах масиву від **L+1** до **R**, порівнюємо їх з **Min**(мінімальним) та **Max**(максимальним) елементами. Обмінюємо їх місцями у випадку, якщо порівнюваний елемент перевищує максимум або менший за мінімум.
4. Допоки виконується умова ($L < R$) повторюємо пункти 1-3.

Загальна схема проходів сортування:



Результат:

	0	1	2	3	4	5	n-1=6
A	4	6	8	11	15	17	18

Для реалізації використовуються наступні змінні:

N - довжина (кількість елементів) масиву **A**;

A - масив, що сортується;

L – індекс лівої границі;

R - індекс правої границі;

Min – індекс мінімального елемента який ставиться на позицію **A[L]**;

Max – індекс максимального елемента який ставиться на позицію **A[R]**;

i – індекс поточного елемента в діапазоні від **L+1** до **R**, масиву **A** , взятого для порівняння;

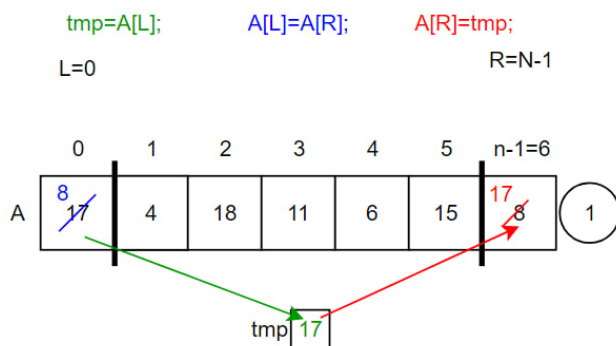
tmp – тимчасова комірка для обміну місцями двох елементів масиву.

Схема одного проходу гібридного алгоритму "вибір№3 – обмін".

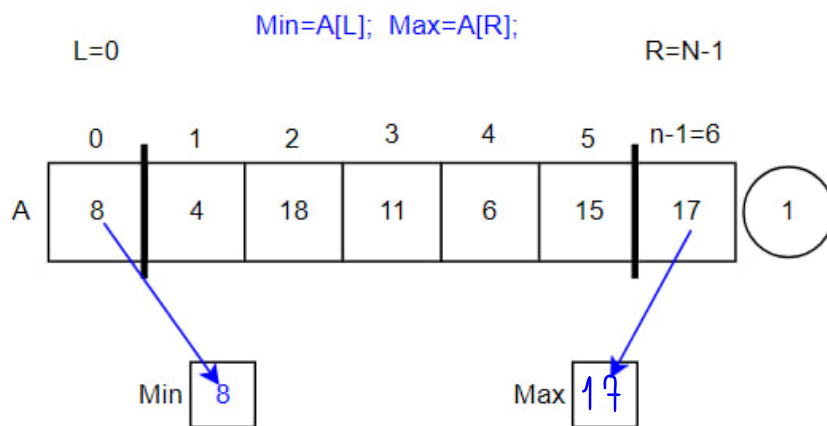
Демонстрація роботи алгоритму з першого крок.

Крок 1. Початкові установки змінних $L = 0, R = N-1$;

Перевірка виконання умови ($A[L] > A[R]$). Умова справджується, тому виконується тіло умовної конструкції.

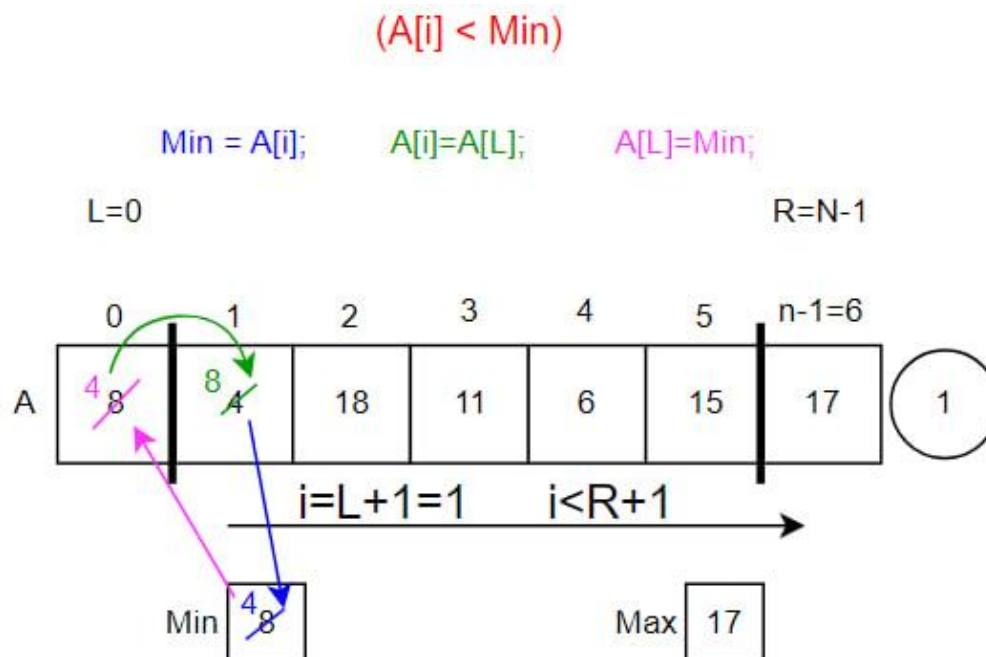


Крок 2. Ініціалізуються змінні **Min** та **Max**.

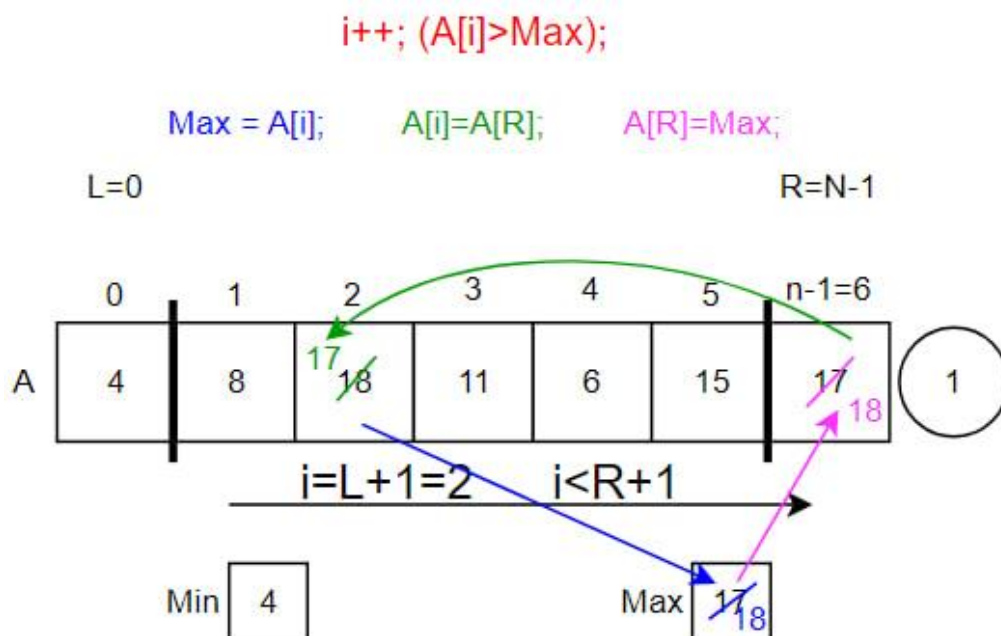


Крок 3. Починається цикл з індексом i , від $L+1$ до $R+1$.

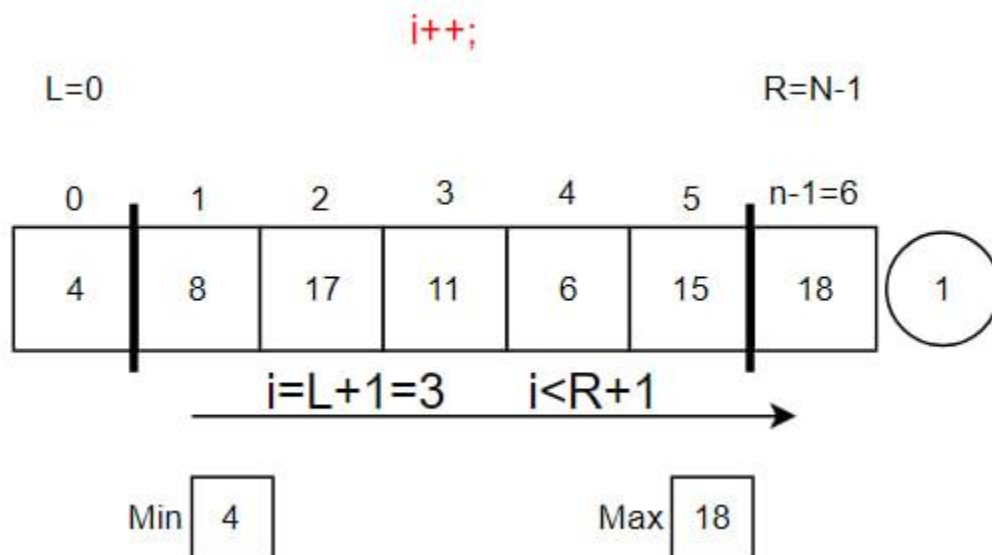
Перевіряється умова $(A[i] < \text{Min})$, вона справджується тому виконується тіло умовної конструкції.



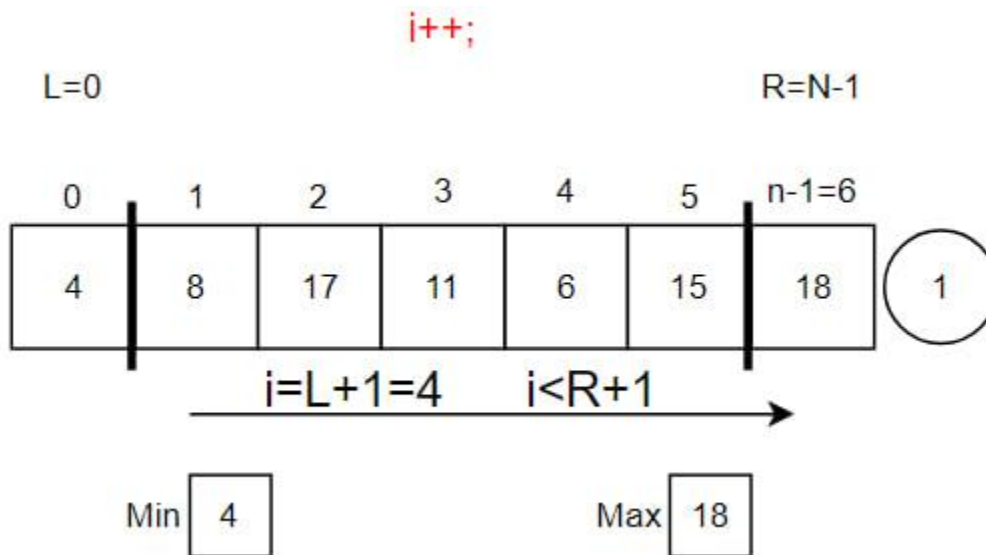
Крок 4. Лічильник i рухається далі, $i++$. Умова $(A[i] < \text{Min})$ не справджується, перевіряється наступна умова $(A[i] > \text{Max})$. Вона справджується, виконується тіло умовної конструкції на встановлення максимального елемента.



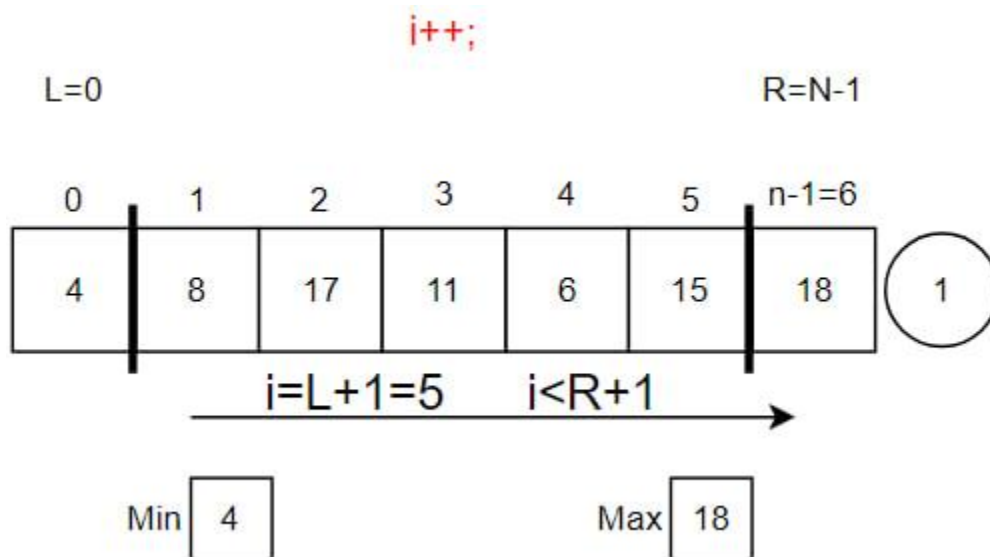
Крок 5. Лічильник i рухається далі, $i++$. Умова $(A[i] < \text{Min})$ не справджується, перевіряється наступна умова $(A[i] > \text{Max})$. Наступна умова теж не справджується. Цикл іде на наступну ітерацію.



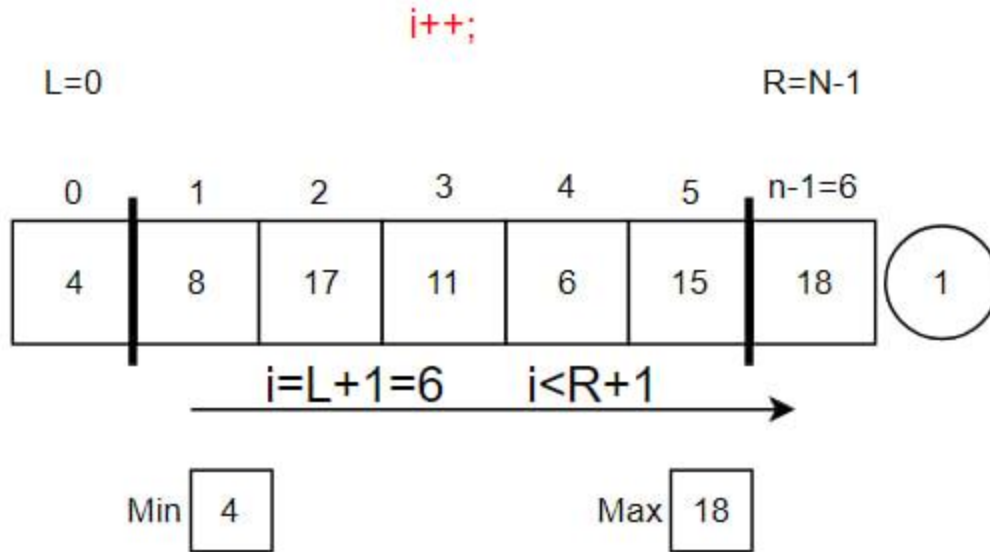
Крок 6. Лічильник i рухається далі, $i++$. Умова $(A[i] < \text{Min})$ не справджується, перевіряється наступна умова $(A[i] > \text{Max})$. Наступна умова теж не справджується. Цикл іде на наступну ітерацію.



Крок 7. Лічильник i рухається далі, $i++$. Умова $(A[i] < \text{Min})$ не справджується, перевіряється наступна умова $(A[i] > \text{Max})$. Наступна умова теж не справджується. Цикл іде на наступну ітерацію.

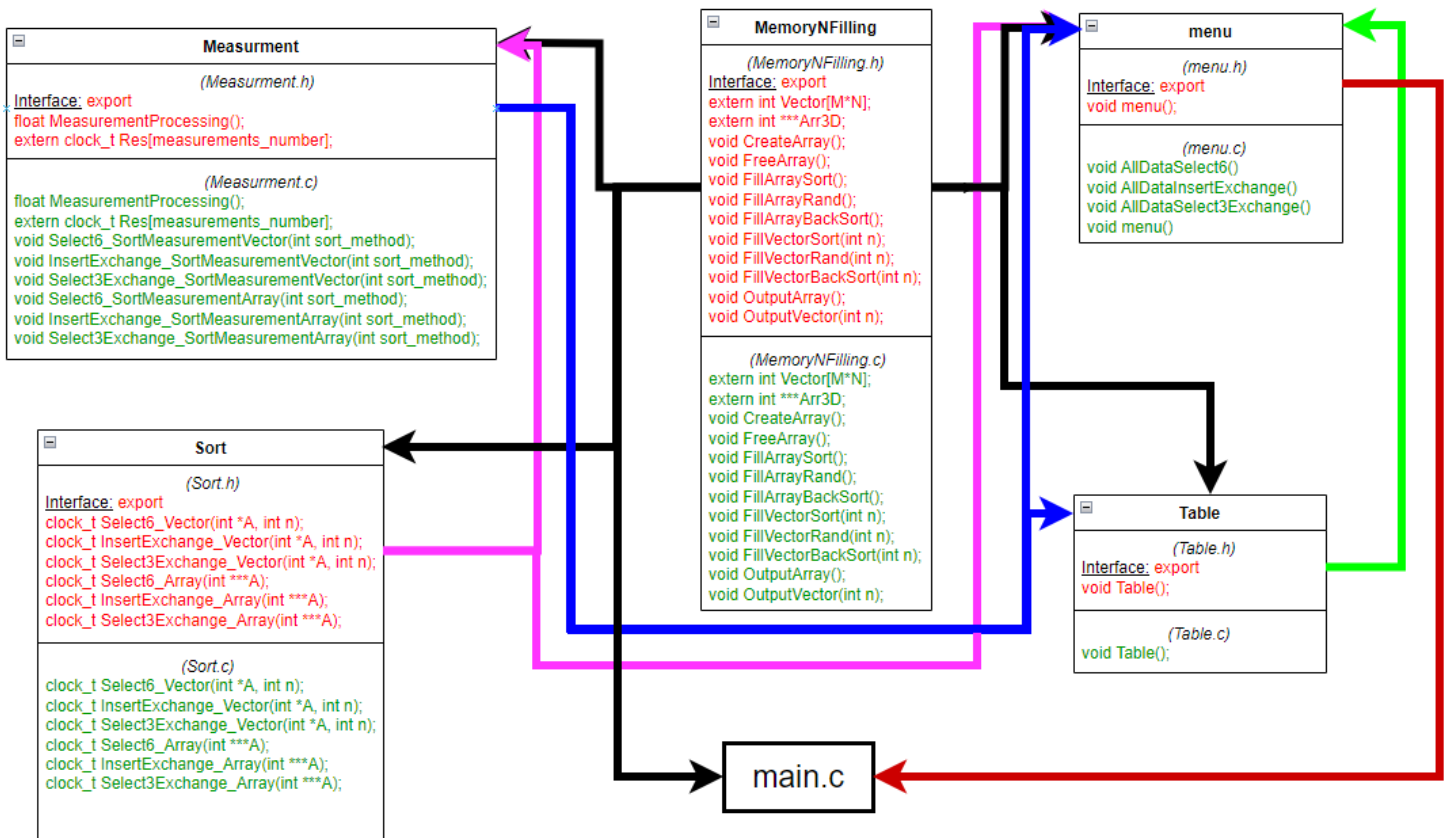


Крок 8. Лічильник i рухається далі, $i++$. Умова $(A[i] < \text{Min})$ не справджується, перевіряється наступна умова $(A[i] > \text{Max})$. Наступна умова теж не справджується. Цикл іде на наступну ітерацію.



Крок 9. Лічильник i закінчує прохід. Алгоритм іде на наступну ітерацію, $L=L+1$, $R=R-1$;

Схема взаємозв'язків модулів і процедур



Опис призначення процедур та функцій

1. **Main** – початкова точка програми, виклик меню.

2. **Модуль menu** – призначений для глобального об'явлення змінних і масивів, а також інтерфейсні функції програми.

- I. **menu()** – безтипова функція, початковий інтерфейс вибору режиму роботи: тестування кожного алгоритму для векторів та матриць, вивід таблиці виміру часу.
- II. **AllDataSelect6()** - безтипова функція, інтерфейсу вибору впорядкованості масиву та вектора для алгоритму Select6.
- III. **AllDataInsertExchange()** - безтипова функція, інтерфейсу вибору впорядкованості масиву та вектора для алгоритму InsertExchange.
- IV. **AllDataSelect3Exchange()** - безтипова функція, інтерфейсу вибору впорядкованості масиву та вектора для алгоритму Select3Exchange.

3. **Модуль MemoryNfilling** – створення ,вивід та заповнення масивів.

- I. **void CreateArray()** - функція створення динамічного тривимірного масиву.
- II. **void FreeArray()** – функція вивільнення пам'яті динамічного тривимірного масиву.
- III. **void FillArraySort()** – Заповнення динамічного тривимірного масиву впорядковано.
- IV. **void FillArrayRand()** - Заповнення динамічного тривимірного масиву випадково.
- V. **void FillArrayBackSort()** - Заповнення динамічного тривимірного масиву Обернено-впорядковано.
- VI. **void FillVectorSort(int n)** – Заповнення вектора впорядковано, як параметр приймається кількість елементів що заповнюються.
- VII. **void FillVectorRand(int n)** - Заповнення вектора випадково, як параметр приймається кількість елементів що заповнюються.

- VIII. **void FillVectorBackSort(int n)** – Заповнення вектора обернено-впорядковано, як параметр приймається кількість елементів що заповнюються.
- IX. **void OutputArray()** – Вивід значень динамічного тривимірного масиву.
- X. **void OutputVector(int n)** - Вивід значень вектора, як параметр приймається кількість значень які треба вивести.

4. Модуль Measurment – призначений для виміру роботи алгоритмів сортування.

- I. **void Select6_SortMeasurementVector(int sort_method)** – функція яка заповнює масив Res значеннями виміру часу роботи алгоритму Select6 при сортуванні вектора. Як параметр приймається режим заповнення вектора.
- II. **void InsertExchange_SortMeasurementVector(int sort_method)** - функція яка заповнює масив Res значеннями виміру часу роботи алгоритму InsertExchange при сортуванні вектора. Як параметр приймається режим заповнення вектора.
- III. **void Select3Exchange_SortMeasurementVector(int sort_method)** - функція яка заповнює масив Res значеннями виміру часу роботи алгоритму Select3Exchange при сортуванні вектора. Як параметр приймається режим заповнення вектора.
- IV. **void Select6_SortMeasurementArray(int sort_method)** - функція яка заповнює масив Res значеннями виміру часу роботи алгоритму Select6 при сортуванні трьохвимірної матриці. Як параметр приймається режим заповнення трьохвимірної матриці.
- V. **void InsertExchange_SortMeasurementArray(int sort_method)** - функція яка заповнює масив Res значеннями виміру часу роботи алгоритму InsertExchange при сортуванні трьохвимірної матриці. Як параметр приймається режим заповнення трьохвимірної матриці.

- VI. **void Select3Exchange_SortMeasurementArray(int sort_method)** - функція яка заповнює масив Res значеннями виміру часу роботи алгоритму Select3Exchange при сортуванні трьохвимірної матриці. Як параметр приймається режим заповнення трьохвимірної матриці.
- VII. **float MeasurementProcessing()** - безтипова функція, яка оброблює результати вимірів, тобто відкидає перші виміри, а також максимальний та мінімальний час роботи алгоритму.

5. Модуль Sort містить функцій призначенні для сортування масиву та вектора.

- I. **clock_t Select6_Vector(int *A, int n)** – Алгоритм сортування №6 методу прямого вибору за не зменшенням для вектора, як параметр приймає вектор який необхідно відсортувати та розмір цього вектора.
- II. **clock_t InsertExchange_Vector(int *A, int n)** - Гібридний алгоритм "вставка – обмін" для сортування за не зменшенням вектора, як параметр приймає вектор який необхідно відсортувати та розмір цього вектора.
- III. **clock_t Select3Exchange_Vector(int *A, int n)** - Гібридний алгоритм сортування "вибір№3 – обмін" за не зменшенням для вектора, як параметр приймає вектор який необхідно відсортувати та розмір цього вектора.
- IV. **clock_t Select6_Array(int ***A)** - Алгоритм сортування №6 методу прямого вибору для, наскрізного сортування по стовпцям за не зменшенням масиву, як параметр приймає трьохвимірний масив який необхідно відсортувати.
- V. **clock_t InsertExchange_Array(int ***A)** - Гібридний алгоритм "вставка – обмін" для, наскрізного сортування по стовпцям за

не зменшенням масиву, як параметр приймає трьохвимірний масив який необхідно відсортувати.

- VI. **clock_t Select3Exchange_Array(int ***A)** - Гібридний алгоритм "вибірN^o3 – обмін" для, наскрізного сортування по стовпцям за не зменшенням масиву, як параметр приймає трьохвимірний масив який необхідно відсортувати.

6. Модуль Table містить функцію призначену для виведення таблиці результатів виміру часу сортування для масивів та векторів для всіх алгоритмів.

- I. **void Table()** – Функція для формування та виводу таблиці результатів виміру часу сортування для масивів та векторів для всіх трьох алгоритмів.

Код програми

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "menu.h"
#include "MemoryNfilling.h"

int main() //Головна функція, викликає меню
{
    srand(time(NULL));
    menu();
    return 0;
}
```

menu.h

```
#ifndef MENU_H_INCLUDED
#define MENU_H_INCLUDED

void menu(); //Меню вибору алгоритмів сортування для демонстрації сортування
вектора та масива та перегляду повної таблиці виміру.

#endif // MENU_H_INCLUDED
```

menu.c

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "Measurement.h"
#include "MemoryNfilling.h"
#include "Sort.h"
#include "Table.h"
#include "menu.h"

const int modes_number = 5;
const char menu_items[5][20] = {"1.Select6",
"2.InsertExchange", "3.Select3Exchange", "4.Table", "5.Exit"};
const int modes_sorting = 4;
const char modes_sorting_items[4][20] = {"1.BackOrdered", "2.Random",
"3.Ordered", "4.Back to menu"};
void menu() //Меню вибору алгоритмів сортування для демонстрації сортування
вектора та масива та перегляду повної таблиці виміру.
{
    while(1)
    {
        fflush(stdin);
        int current_mode;
        system("cls");
        printf("CourseWork by: Polishchuk Mariia KV-33\n\n");
        printf("Size P:%d M:%d N:%d\n\n", P, M, N);
        for(int i = 0; i < modes_number; i++)
        {
            printf("%s\n", menu_items[i]);
        }
        printf("\nInput modes number and press ENTER:");
        scanf("%d", &current_mode);
        switch(current_mode)
        {
            case 1:
                AllDataSelect6();
                break;

            case 2:
                AllDataInsertExchange();
                break;

            case 3:
                AllDataSelect3Exchange();
                break;

            case 4:
                system("cls");
                fflush(stdin);
                Table();
                break;

            case 5:
                system("cls");
                exit(1);

            default:
                printf("\nInvalid value entered! Please try again.");
                Sleep(2000);
                fflush(stdin);
        }
    }
}

```

```

    }
}

void AllDataSelect6() //Меню вибору режиму заповнення вектора та масива для
сортювання алгоритмом сортювання №6 методу прямого вибору. Демонстрація
результатів.
{
    int select6_mode;
    while(1)
    {
        fflush(stdin);
        system("cls");
        printf("Demonstration of Select6 algorithm\n\n");
        for(int i = 0; i < modes_sorting; i++)
        {
            printf("%s\n", modes_sorting_items[i]);
        }
        printf("Input modes number and press ENTER:");
        scanf("%d", &select6_mode);
        switch(select6_mode)
        {
            case 1:
                fflush(stdin);
                system("cls");
                printf("Demonstration of Select6 algorithm on BackOrdered Array
and Vector\n\n");
                printf("Size P:%d M:%d N:%d\n", P, M, N);
                printf("\nVector before sorting:\n");
                FillVectorBackSort(M*N);
                OutputVector(M*N);
                Select6_Vector(Vector, M*N);
                printf("\nVector after sorting:\n");
                OutputVector(M*N);
                CreateArray();
                printf("\n\nArray before sorting:\n");
                FillArrayBackSort();
                OutputArray();
                Select6_Array(Arr3D);
                printf("\nArray after sorting:\n");
                OutputArray();
                FreeArray();
                getchar();
                break;
            case 2:
                fflush(stdin);
                system("cls");
                printf("Demonstration of Select6 algorithm on Random Array and
Vector\n\n");
                printf("Size P:%d M:%d N:%d\n", P, M, N);
                printf("\nVector before sorting:\n");
                FillVectorRand(M*N);
                OutputVector(M*N);
                Select6_Vector(Vector, M*N);
                printf("\nVector after sorting:\n");
                OutputVector(M*N);
                CreateArray();
                printf("\n\nArray before sorting:\n");

```

```

        FillArrayRand();
        OutputArray();
        Select6_Array(Arr3D);
        printf("\nArray after sorting:\n");
        OutputArray();
        FreeArray();
        getchar();
        break;
    case 3:
        fflush(stdin);
        system("cls");
        printf("Demonstration of Select6 algorithm on Sorted Array and
Vector\n\n");
        printf("Size P:%d M:%d N:%d\n", P, M, N);
        printf("\nVector before sorting:\n");
        FillVectorSort(M*N);
        OutputVector(M*N);
        Select6_Vector(Vector, M*N);
        printf("\nVector after sorting:\n");
        OutputVector(M*N);
        CreateArray();
        printf("\n\nArray before sorting:\n");
        FillArraySort();
        OutputArray();
        Select6_Array(Arr3D);
        printf("\nArray after sorting:\n");
        OutputArray();
        FreeArray();
        getchar();
        break;
    case 4:
        menu();
        break;
    default:
        printf("\nInvalid value entered! Please try again.");
        Sleep(2000);
        fflush(stdin);
    }
}
}

```

void AllDataInsertExchange() //Меню вибору режиму заповнення вектора та масива для сортування гібридним алгоритмом "вставка - обмін". Демонстрація результатів.

```

{
    int InsertExchange_mode;
    while (1)
    {
        fflush(stdin);
        system("cls");
        printf("Demonstration of InsertExchange algorithm\n\n");
        for (int i = 0; i < modes_sorting; i++)
        {
            printf("%s\n", modes_sorting_items[i]);
        }
        printf("Input modes number and press ENTER:");
        scanf("%d", &InsertExchange_mode);
    }
}

```



```

switch(InsertExchange_mode)
{
case 1:
    fflush(stdin);
    system("cls");
    printf("Demonstration of InsertExchange algorithm on BackOrdered
Array and Vector\n\n");
    printf("Size P:%d M:%d N:%d\n", P, M, N);
    printf("\nVector before sorting:\n");
    FillVectorBackSort(M*N);
    OutputVector(M*N);
    InsertExchange_Vector(Vector, M*N);
    printf("\nVector after sorting:\n");
    OutputVector(M*N);
    CreateArray();
    printf("\n\nArray before sorting:\n");
    FillArrayBackSort();
    OutputArray();
    InsertExchange_Array(Arr3D);
    printf("\nArray after sorting:\n");
    OutputArray();
    FreeArray();
    getchar();
    break;
case 2:
    fflush(stdin);
    system("cls");
    printf("Demonstration of InsertExchange algorithm on Random Array
and Vector\n\n");
    printf("Size P:%d M:%d N:%d\n", P, M, N);
    printf("\nVector before sorting:\n");
    FillVectorRand(M*N);
    OutputVector(M*N);
    InsertExchange_Vector(Vector, M*N);
    printf("\nVector after sorting:\n");
    OutputVector(M*N);
    CreateArray();
    printf("\n\nArray before sorting:\n");
    FillArrayRand();
    OutputArray();
    InsertExchange_Array(Arr3D);
    printf("\nArray after sorting:\n");
    OutputArray();
    FreeArray();
    getchar();
    break;
case 3:
    fflush(stdin);
    system("cls");
    printf("Demonstration of InsertExchange algorithm on Sorted Array
and Vector\n\n");
    printf("Size P:%d M:%d N:%d\n", P, M, N);
    printf("\nVector before sorting:\n");
    FillVectorSort(M*N);
    OutputVector(M*N);
    InsertExchange_Vector(Vector, M*N);
    printf("\nVector after sorting:\n");

```

```

        OutputVector(M*N);
        CreateArray();
        printf("\n\nArray before sorting:\n");
        FillArraySort();
        OutputArray();
        InsertExchange_Array(Arr3D);
        printf("\nArray after sorting:\n");
        OutputArray();
        FreeArray();
        getchar();
        break;
    case 4:
        menu();
        break;
    default:
        printf("\nInvalid value entered! Please try again.");
        Sleep(2000);
        fflush(stdin);
    }
}

}

void AllDataSelect3Exchange() //Меню вибору режиму заповнення вектора та
масива для сортування гібридним алгоритмом "вибір№3 - обмін". Демонстрація
результатів.
{
    int Select3Exchange_mode;
    while(1)
    {
        fflush(stdin);
        system("cls");
        printf("Demonstration of Select3Exchange algorithm\n\n");
        for(int i = 0; i < modes_sorting; i++)
        {
            printf("%s\n", modes_sorting_items[i]);
        }
        printf("Input modes number and press ENTER:");
        scanf("%d", &Select3Exchange_mode);
        switch(Select3Exchange_mode)
        {
            case 1:
                fflush(stdin);
                system("cls");
                printf("Demonstration of Select3Exchange algorithm on BackOrdered
Array and Vector\n\n");
                printf("Size P:%d M:%d N:%d\n", P, M, N);
                printf("\nVector before sorting:\n");
                FillVectorBackSort(M*N);
                OutputVector(M*N);
                Select3Exchange_Vector(Vector, M*N);
                printf("\nVector after sorting:\n");
                OutputVector(M*N);
                CreateArray();
                printf("\n\nArray before sorting:\n");
                FillArrayBackSort();
                OutputArray();
                Select3Exchange_Array(Arr3D);

```

```

        printf("\nArray after sorting:\n");
        OutputArray();
        FreeArray();
        getchar();
        break;
    case 2:
        fflush(stdin);
        system("cls");
        printf("Demonstration of Select3Exchange algorithm on Random
Array and Vector\n\n");
        printf("Size P:%d M:%d N:%d\n", P, M, N);
        printf("\nVector before sorting:\n");
        FillVectorRand(M*N);
        OutputVector(M*N);
        Select3Exchange_Vector(Vector, M*N);
        printf("\nVector after sorting:\n");
        OutputVector(M*N);
        CreateArray();
        printf("\n\nArray before sorting:\n");
        FillArrayRand();
        OutputArray();
        Select3Exchange_Array(Arr3D);
        printf("\nArray after sorting:\n");
        OutputArray();
        FreeArray();
        getchar();
        break;
    case 3:
        fflush(stdin);
        system("cls");
        printf("Demonstration of Select3Exchange algorithm on Sorted
Array and Vector\n\n");
        printf("Size P:%d M:%d N:%d\n", P, M, N);
        printf("\nVector before sorting:\n");
        FillVectorSort(M*N);
        OutputVector(M*N);
        Select3Exchange_Vector(Vector, M*N);
        printf("\nVector after sorting:\n");
        OutputVector(M*N);
        CreateArray();
        printf("\n\nArray before sorting:\n");
        FillArraySort();
        OutputArray();
        Select3Exchange_Array(Arr3D);
        printf("\nArray after sorting:\n");
        OutputArray();
        FreeArray();
        getchar();
        break;
    case 4:
        menu();
        break;
    default:
        printf("\nInvalid value entered! Please try again.");
        Sleep(2000);
        fflush(stdin);
}

```

```

    }
}

```

Sort.h

```

#ifndef SORT_H_INCLUDED
#define SORT_H_INCLUDED
#include <time.h>

clock_t Select6_Vector(int *A, int n); //Алгоритм сортування №6 методу
прямого вибору вектора за незменшенням.
clock_t InsertExchange_Vector(int *A, int n); //Гібридний алгоритм "вставка -
обмін" для сортування вектора за незменшенням.
clock_t Select3Exchange_Vector(int *A, int n); //Гібридний алгоритм "вибір№3
- обмін" для сортування вектора за незменшенням.
clock_t Select6_Array(int ***A); //Алгоритм №6 методу прямого вибору для
сортування окремо кожного перерізу тривимірного масива наскрізно по
стовпчиках за незменшенням.
clock_t InsertExchange_Array(int ***A); //Гібридний алгоритм "вставка -
обмін" для сортування окремо кожного перерізу тривимірного масива наскрізно
по стовпчиках за незменшенням.
clock_t Select3Exchange_Array(int ***A); //Гібридний алгоритм "вибір№3 -
обмін" для сортування окремо кожного перерізу тривимірного масива наскрізно
по стовпчиках за незменшенням.

#endif // SORT_H_INCLUDED

```

Sort.c

```

#include <stdio.h>
#include <stdlib.h>
#include "Sort.h"
#include <time.h>
#include "MemoryNfilling.h"

clock_t Select6_Vector(int *A, int n) //Алгоритм сортування №6 методу прямого
вибору вектора за незменшенням.
{
    int imin, tmp;
    clock_t time_start, time_stop;
    time_start = clock();
    for(int s=0; s<n-1; s++)
    {
        imin=s;
        for(int i=s+1; i<n; i++)
            if (A[i]<A[imin]) imin=i;
        if (imin!=s)
        {
            tmp=A[imin];
            A[imin]=A[s];
            A[s]=tmp;
        }
    }
    time_stop = clock();
    return time_stop - time_start;
}

```

clock_t InsertExchange_Vector(int *A, int n) //Гібридний алгоритм "вставка - обмін" для сортування вектора за незменшенням.

```
{
    int j, tmp;
    clock_t time_start, time_stop;
    time_start = clock();
    for(int i=1; i<n; i++)
    {
        j=i;
        while (j>0 && A[j]<A[j-1])
        {
            tmp=A[j];
            A[j]=A[j-1];
            A[j-1]=tmp;
            j=j-1;
        }
    }
    time_stop = clock();
    return time_stop - time_start;
}
```

clock_t Select3Exchange_Vector(int *A, int n) //Гібридний алгоритм "вибір№3 - обмін" для сортування вектора за незменшенням.

```
{
    int Min, Max, tmp;
    int L, R;
    clock_t time_start, time_stop;
    time_start = clock();
    L=0;
    R=n-1;
    while (L<R)
    {
        if (A[L] > A[R])
        {
            tmp=A[L];
            A[L]=A[R];
            A[R]=tmp;
        }
        Min=A[L];
        Max=A[R];
        for(int i=L+1; i<R+1; i++)
        {
            if (A[i] < Min)
            {
                Min=A[i];
                A[i]=A[L];
                A[L]=Min;
            }
            else if (A[i] > Max)
            {
                Max=A[i];
                A[i]=A[R];
                A[R]=Max;
            }
        }
        L=L+1;
        R=R-1;
    }
}
```

```

    }
    time_stop = clock();
    return time_stop - time_start;
}

```

clock_t Select6_Array(int ***A) //Алгоритм №6 методу прямого вибору для сортування окремо кожного перерізу тривимірного масива наскрізно по стовпчиках за незменшенням.

```

{
    int imin, jmin, tmp;
    clock_t time_start, time_stop;
    time_start = clock();
    for(int dem = 0; dem <= P-1; dem++)
    {
        for(int row = 0; row <= N-1; row++)
        {
            for(int col = 0; col <= M-1; col++)
            {
                imin = row;
                jmin = col;
                for(int j = col + 1; j <= M-1; j++)
                {
                    if(A[dem][j][row] < A[dem][jmin][row])
                    {
                        jmin = j;
                    }
                }
                for(int i = row + 1; i <= N-1; i++)
                {
                    for(int j = 0; j <= M-1; j++)
                    {
                        if(A[dem][j][i] < A[dem][jmin][imin])
                        {
                            jmin = j;
                            imin = i;
                        }
                    }
                }
                if(jmin != col || imin != row)
                {
                    tmp = A[dem][jmin][imin];
                    A[dem][jmin][imin] = A[dem][col][row];
                    A[dem][col][row] = tmp;
                }
            }
        }
    }
    time_stop = clock();
    return time_stop - time_start;
}

```

clock_t InsertExchange_Array(int ***A) //Гібридний алгоритм "вставка - обмін" для сортування окремо кожного перерізу тривимірного масива наскрізно по стовпчиках за незменшенням.

```

{
    int j, i, prev_j, prev_i, tmp;

```

```

clock_t time_start, time_stop;
time_start = clock();
for(int dem = 0; dem <= P-1; dem++)
{
    for(int row = 0; row <= N-1; row++)
    {
        for(int col = 0; col <= M-1; col++)
        {
            j = col;
            i = row;
            if(j != 0)
            {
                prev_j = j - 1;
                prev_i = i;
            }
            else
            {
                prev_j = M-1;
                prev_i = i - 1;
            }
            while((j > 0 || i != 0) && A[dem][j][i] <
A[dem][prev_j][prev_i])
            {
                tmp = A[dem][j][i];
                A[dem][j][i] = A[dem][prev_j][prev_i];
                A[dem][prev_j][prev_i] = tmp;
                if(j != 0)
                {
                    j--;
                }
                else
                {
                    j = M-1;
                    i--;
                }
                if(prev_j != 0)
                {
                    prev_j--;
                }
                else
                {
                    prev_j = M-1;
                    prev_i--;
                }
            }
        }
    }
}
time_stop = clock();
return time_stop - time_start;
}

```

```

clock_t Select3Exchange_Array(int ***A) //Гібридний алгоритм "вибір№3 -
обмін" для сортування окремо кожного перерізу тривимірного масива наскрізно
по стовпчиках за незменшенням.
{

```

```

int Min, Max, tmp;
clock_t time_start, time_stop;
time_start = clock();
for(int dem = 0; dem <= P-1; dem++)
{
    for(int row = 0; row <= (N/2)-1; row++)
    {
        for(int col = 0; col <= M-1; col++)
        {
            if(A[dem][col][row] > A[dem][M - 1 - col][N - 1 - row])
            {
                tmp = A[dem][col][row];
                A[dem][col][row] = A[dem][M - 1 - col][N - 1 - row];
                A[dem][M - 1 - col][N - 1 - row] = tmp;
            }
            Min = A[dem][col][row];
            Max = A[dem][M - 1 - col][N - 1 - row];
            for(int i = col + 1; i <= M - 1; i++)
            {
                if(A[dem][i][row] < Min)
                {
                    Min = A[dem][i][row];
                    A[dem][i][row] = A[dem][col][row];
                    A[dem][col][row] = Min;
                }
                else if(A[dem][i][row] > Max)
                {
                    Max = A[dem][i][row];
                    A[dem][i][row] = A[dem][M - 1 - col][N - 1 - row];
                    A[dem][M - 1 - col][N - 1 - row] = Max;
                }
            }
        }
        for(int i = row + 1; i < N - 1 - row; i++)
        {
            for(int j = 0; j <= M - 1; j++)
            {
                if(A[dem][j][i] < Min)
                {
                    Min = A[dem][j][i];
                    A[dem][j][i] = A[dem][col][row];
                    A[dem][col][row] = Min;
                }
                else if(A[dem][j][i] > Max)
                {
                    Max = A[dem][j][i];
                    A[dem][j][i] = A[dem][M - 1 - col][N - 1 - row];
                    A[dem][M - 1 - col][N - 1 - row] = Max;
                }
            }
        }
        for(int i = col + 1; i <= M - 1; i++)
        {
            if(A[dem][M - 1 - i][N - 1 - row] < Min)
            {
                Min = A[dem][M - 1 - i][N - 1 - row];
                A[dem][M - 1 - i][N - 1 - row] = A[dem][col][row];
                A[dem][col][row] = Min;
            }
        }
    }
}

```



```

    }
    if(A[dem][M - 1 - i][N - 1 - row] > Max)
    {
        Max = A[dem][M - 1 - i][N - 1 - row];
        A[dem][M - 1 - i][N - 1 - row] = A[dem][M - 1 -
col][N - 1 - row];
        A[dem][M - 1 - col][N - 1 - row] = Max;
    }
}
}
}
for(int col = 0; col <= (M-1)/2; col++)
{
    if(A[dem][col][(N-1)/2] > A[dem][M - 1 - col][(N-1)/2])
    {
        tmp = A[dem][col][(N-1)/2];
        A[dem][col][(N-1)/2] = A[dem][M - 1 - col][(N-1)/2];
        A[dem][M - 1 - col][(N-1)/2] = tmp;
    }
    Min = A[dem][col][(N-1)/2];
    Max = A[dem][M - 1 - col][(N-1)/2];
    for(int i = col + 1; i <= M - 1 - col; i++)
    {
        if(A[dem][i][(N-1)/2] < Min)
        {
            Min = A[dem][i][(N-1)/2];
            A[dem][i][(N-1)/2] = A[dem][col][(N-1)/2];
            A[dem][col][(N-1)/2] = Min;
        }
        else if(A[dem][i][(N-1)/2] > Max)
        {
            Max = A[dem][i][(N-1)/2];
            A[dem][i][(N-1)/2] = A[dem][M - 1 - col][(N-1)/2];
            A[dem][M - 1 - col][(N-1)/2] = Max;
        }
    }
}
}
time_stop = clock();
return time_stop - time_start;
}

```

Measurment.h

```

#ifndef MEASUREMENT_H_INCLUDED
#define MEASUREMENT_H_INCLUDED
#include <time.h>

// Загальна кількість вимірів часу роботи алгоритма
#define measurements_number 28
// Кількість відкинутих початкових вимірів
#define rejected_number 2
// Кількість відкинутих вимірів з мінімальними значеннями.
// Вона ж дорівнює кількості відкинутих вимірів
// з максимальними значеннями.
#define min_max_number 3
// Масив значень часу роботи алгоритма

```

```
extern clock_t Res[measurements_number];
// Функція обробки і усереднення значень вимірів
// часу роботи алгоритма.
// Повертає усереднене значення часу роботи алгоритма.
float MeasurementProcessing();
```

```
#endif // MEASUREMENT_H_INCLUDED
```

Measurment.c

```
#include "Measurement.h"
#include "MemoryNfilling.h"
#include "Sort.h"
#include <stdio.h>
```

```
clock_t Res[measurements_number];
```

```
float MeasurementProcessing() //Функція обробки і усереднення значень вимірів
часу роботи алгоритма.
```

```
{
    long int Sum;
    float AverageValue;
    clock_t buf;
    int L = rejected_number, R = measurements_number - 1;
    int k = rejected_number;
    for (int j=0; j < min_max_number; j++)
    {
        for (int i = L; i < R; i++)
        {
            if (Res[i] > Res[i + 1])
            {
                buf = Res[i];
                Res[i] = Res[i + 1];
                Res[i + 1] = buf;
                k = i;
            }
        }
        R = k;
        for (int i = R - 1; i >= L; i--)
        {
            if (Res[i] > Res[i + 1])
            {
                buf = Res[i];
                Res[i] = Res[i + 1];
                Res[i + 1] = buf;
                k = i;
            }
        }
        L = k + 1;
    }
    Sum=0;
    for (int i = rejected_number + min_max_number; i < measurements_number -
min_max_number; i++)
        Sum = Sum + Res[i];
    AverageValue = (float) Sum / (float) (measurements_number - 2*min_max_number
- rejected_number);
    printf("");
}
```

```

        return AverageValue;
    }

void Select6_SortMeasurementVector(int sort_method) //функція яка заповнює
масив Res значеннями виміру часу роботи алгоритма Select6 при сортуванні
вектора.
{
    for (int i = 0; i < measurements_number; i++)
    {
        switch(sort_method)
        {
            case 1:
                FillVectorSort(M*N);
                break;
            case 2:
                FillVectorRand(M*N);
                break;
            case 3:
                FillVectorBackSort(M*N);
                break;
        }
        Res[i] = Select6_Vector(Vector, M*N);
    }
}

void InsertExchange_SortMeasurementVector(int sort_method) //функція яка
заповнює масив Res значеннями виміру часу роботи алгоритма InsertExchange при
сортуванні вектора.
{
    for (int i = 0; i < measurements_number; i++)
    {
        switch(sort_method)
        {
            case 1:
                FillVectorSort(M*N);
                break;
            case 2:
                FillVectorRand(M*N);
                break;
            case 3:
                FillVectorBackSort(M*N);
                break;
        }
        Res[i] = InsertExchange_Vector(Vector, M*N);
    }
}

void Select3Exchange_SortMeasurementVector(int sort_method) //функція яка
заповнює масив Res значеннями виміру часу роботи алгоритма Select3Exchange
при сортуванні вектора.
{
    for (int i = 0; i < measurements_number; i++)
    {
        switch(sort_method)
        {
            case 1:

```

```

        FillVectorSort(M*N);
        break;
    case 2:
        FillVectorRand(M*N);
        break;
    case 3:
        FillVectorBackSort(M*N);
        break;
    }
    Res[i] = Select3Exchange_Vector(Vector, M*N);
}
}

```

void Select6_SortMeasurementArray(**int** sort_method) //функція яка заповнює масив Res значеннями виміру часу роботи алгоритма Select6 при сортуванні трьохвимірної матриці.

```

{
    for (int i = 0; i < measurements_number; i++)
    {
        CreateArray();
        switch(sort_method)
        {
            case 1:
                FillArraySort();
                break;
            case 2:
                FillArrayRand();
                break;
            case 3:
                FillArrayBackSort();
                break;
        }
        Res[i] = Select6_Array(Arr3D);
        FreeArray();
    }
}

```

void InsertExchange_SortMeasurementArray(**int** sort_method) //функція яка заповнює масив Res значеннями виміру часу роботи алгоритма InsertExchange при сортуванні трьохвимірної матриці.

```

{
    for (int i = 0; i < measurements_number; i++)
    {
        CreateArray();
        switch(sort_method)
        {
            case 1:
                FillArraySort();
                break;
            case 2:
                FillArrayRand();
                break;
            case 3:
                FillArrayBackSort();
                break;
        }
        Res[i] = InsertExchange_Array(Arr3D);
    }
}

```

```

        FreeArray();
    }
}

void Select3Exchange_SortMeasurementArray(int sort_method) //функція яка
заповнює масив Res значеннями виміру часу роботи алгоритма Select3Exchange
при сортуванні вектора.
{
    for (int i = 0; i < measurements_number; i++)
    {
        CreateArray();
        switch(sort_method)
        {
            case 1:
                FillArraySort();
                break;
            case 2:
                FillArrayRand();
                break;
            case 3:
                FillArrayBackSort();
                break;
        }
        Res[i] = Select3Exchange_Array(Arr3D);
        FreeArray();
    }
}

```

MemoryNFilling.h

```

#ifndef MEMORYNFFILLING_H_INCLUDED
#define MEMORYNFFILLING_H_INCLUDED

#define P 3
#define M 512
#define N 512

extern int Vector[M*N];
extern int ***Arr3D;

void CreateArray(); //функція створення динамічного тривимірного масиву.
void FreeArray(); //функція вивільнення пам'яті динамічного тривимірного
масиву.
void FillArraySort(); //Заповнення динамічного тривимірного масиву
впорядковано.
void FillArrayRand(); //Заповнення динамічного тривимірного масиву випадково.
void FillArrayBackSort(); //Заповнення динамічного тривимірного масиву
Обернено-впорядковано.
void FillVectorSort(int n); //Заповнення вектора впорядковано.
void FillVectorRand(int n); //Заповнення вектора випадково.
void FillVectorBackSort(int n); //Заповнення вектора обернено-впорядковано.
void OutputArray(); //Вивід значень динамічного тривимірного масиву.
void OutputVector(int n); //Вивід значень вектора.

#endif // MEMORYNFFILLING_H_INCLUDED

```

MemoryNFilling.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "MemoryNFilling.h"

int Vector[M*N];
int ***Arr3D;

void CreateArray() //функція створення динамічного тривимірного масиву.
{
    Arr3D = (int***) malloc(P*sizeof(int**));
    for (int k=0; k<P; k++)
    {
        Arr3D[k] = (int**) malloc(M*sizeof(int*));
        for (int i=0; i<M; i++)
            Arr3D[k][i] = (int*) malloc(N*sizeof(int));
    }
}

void FreeArray() //функція вивільнення пам'яті динамічного тривимірного масиву.
{
    for (int k=0; k<P; k++)
    {
        for (int i=0; i<M; i++)
            free(Arr3D[k][i]);
        free(Arr3D[k]);
    }
    free(Arr3D);
}

void FillArraySort() //Заповнення динамічного тривимірного масиву
впорядковано.
{
    int number=0;
    for (int k=0; k<P; k++)
        for (int j=0; j<N; j++)
            for (int i=0; i<M; i++)
                Arr3D[k][i][j] = number++;
}

void FillArrayRand() //Заповнення динамічного тривимірного масиву випадково.
{
    for (int k=0; k<P; k++)
        for (int j=0; j<N; j++)
            for (int i=0; i<M; i++)
                Arr3D[k][i][j] = rand() % (P*M*N);
}

void FillArrayBackSort() //Заповнення динамічного тривимірного масиву
Обернено-впорядковано.
{
    int number = P*M*N;
    for (int k=0; k<P; k++)
        for (int j=0; j<N; j++)
```

```

        for (int i=0; i<M; i++)
            Arr3D[k][i][j] = number--;
    }

void FillVectorSort(int n) //Заповнення вектора впорядковано.
{
    for (int i = 0; i < n; i++)
    {
        Vector[i] = i;
    }
}

void FillVectorRand(int n) //Заповнення вектора випадково.
{
    for (int i = 0; i < n; i++)
    {
        Vector[i] = rand() % (M*N);
    }
}

void FillVectorBackSort(int n) //Заповнення вектора обернено-впорядковано.
{
    for( int i = 0; i < n; i++)
    {
        Vector[i] = M*N - 1 - i;
    }
}

void OutputArray() //Вивід значень динамічного тривимірного масиву.
{
    for (int i = 0; i < M; i++)
    {
        for (int k = 0; k < P; k++)
        {
            for (int j = 0; j < N; j++)
            {
                printf("%4d", Arr3D[k][i][j]);
            }
            printf("|");
        }
        printf("\n");
    }
}

void OutputVector(int n) //Вивід значень вектора.
{
    for(int i = 0; i < n; i++)
    {
        printf("%d ", Vector[i]);
    }
}

```

Table.h

```

#ifndef TABLE_H_INCLUDED
#define TABLE_H_INCLUDED

void Table(); //Функція для формування та виводу таблиці результатів виміру.

#endif // TABLE_H_INCLUDED

```

Table.c

```

#include <stdio.h>
#include "Table.h"
#include "Measurement.h"
#include "MemoryNfilling.h"

void Table() //Функція для формування та виводу таблиці результатів виміру.
{
    float ordered, random, backordered;
    printf("Vector\n");
    printf("Size M*N:%d\n", M*N);
    printf("\t\t Ordered \t Random \t BackOrdered \n");
    Select6_SortMeasurementVector(1);
    ordered = MeasurementProcessing();
    Select6_SortMeasurementVector(2);
    random = MeasurementProcessing();
    Select6_SortMeasurementVector(3);
    backordered = MeasurementProcessing();
    printf("Select6\t\t %f\t %f\t %f\n", ordered, random, backordered);
    InsertExchange_SortMeasurementVector(1);
    ordered = MeasurementProcessing();
    InsertExchange_SortMeasurementVector(2);
    random = MeasurementProcessing();
    InsertExchange_SortMeasurementVector(3);
    backordered = MeasurementProcessing();
    printf("InsertExchange\t %f\t %f\t %f\n", ordered, random, backordered);
    Select3Exchange_SortMeasurementVector(1);
    ordered = MeasurementProcessing();
    Select3Exchange_SortMeasurementVector(2);
    random = MeasurementProcessing();
    Select3Exchange_SortMeasurementVector(3);
    backordered = MeasurementProcessing();
    printf("Select3Exchange\t %f\t %f\t %f\n", ordered, random, backordered);

    printf("\nArray\n");
    printf("Size P:%d M:%d N:%d\n", P, M, N);
    printf("\t\t Ordered \t Random \t BackOrdered \n");
    Select6_SortMeasurementArray(1);
    ordered = MeasurementProcessing();
    Select6_SortMeasurementArray(2);
    random = MeasurementProcessing();
    Select6_SortMeasurementArray(3);
    backordered = MeasurementProcessing();
    printf("Select6\t\t %f\t %f\t %f\n", ordered, random, backordered);
    InsertExchange_SortMeasurementArray(1);
    ordered = MeasurementProcessing();
    InsertExchange_SortMeasurementArray(2);

```



```

random = MeasurementProcessing();
InsertExchange_SortMeasurementArray(3);
backordered = MeasurementProcessing();
printf("InsertExchange\t %f\t %f\t %f\n",ordered, random, backordered);
Select3Exchange_SortMeasurementArray(1);
ordered = MeasurementProcessing();
Select3Exchange_SortMeasurementArray(2);
random = MeasurementProcessing();
Select3Exchange_SortMeasurementArray(3);
backordered = MeasurementProcessing();
printf("Select3Exchange\t %f\t %f\t %f\n",ordered, random, backordered);
getchar();
}

```

Тести

Тести, що демонструють коректну роботу кожного із заданих алгоритмів для кожного випадку початкового заповнення масиву та вектора.

Тести проводилися для таких розмірів тривимірного масиву та вектора:

Масив: P = 2, M = 5, N = 5.

Вектор: M*N = 25.

Select6

1) Демонстрація результатів для алгоритму сортування №6 методу прямого вибору при заповненні впорядковано.

```

D:\University\sda\Kurswork\kurswork\bin\Debug\kurswork.exe
Demonstration of Select6 algorithm on Sorted Array and Vector
Size P:2 M:5 N:5
Vector before sorting:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
Vector after sorting:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
Array before sorting:
 0  5 10 15 20| 25 30 35 40 45|
 1  6 11 16 21| 26 31 36 41 46|
 2  7 12 17 22| 27 32 37 42 47|
 3  8 13 18 23| 28 33 38 43 48|
 4  9 14 19 24| 29 34 39 44 49|
Array after sorting:
 0  5 10 15 20| 25 30 35 40 45|
 1  6 11 16 21| 26 31 36 41 46|
 2  7 12 17 22| 27 32 37 42 47|
 3  8 13 18 23| 28 33 38 43 48|
 4  9 14 19 24| 29 34 39 44 49|

```

2) Демонстрація результатів для алгоритму сортування №6 методу прямого вибору при заповненні обернено-впорядковано.

```
D:\University\sda\Kurswork\kurswork\bin\Debug\kurswork.exe
Demonstration of Select6 algorithm on BackOrdered Array and Vector

Size P:2 M:5 N:5

Vector before sorting:
24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
Vector after sorting:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24

Array before sorting:
50 45 40 35 30| 25 20 15 10 5|
49 44 39 34 29| 24 19 14 9 4|
48 43 38 33 28| 23 18 13 8 3|
47 42 37 32 27| 22 17 12 7 2|
46 41 36 31 26| 21 16 11 6 1|

Array after sorting:
26 31 36 41 46| 1 6 11 16 21|
27 32 37 42 47| 2 7 12 17 22|
28 33 38 43 48| 3 8 13 18 23|
29 34 39 44 49| 4 9 14 19 24|
30 35 40 45 50| 5 10 15 20 25|
```

3) Демонстрація результатів для алгоритму сортування №6 методу прямого вибору при заповненні випадково.

```
D:\University\sda\Kurswork\kurswork\bin\Debug\kurswork.exe
Demonstration of Select6 algorithm on Random Array and Vector

Size P:2 M:5 N:5

Vector before sorting:
8 5 24 6 16 6 11 8 22 10 8 20 10 2 17 8 6 1 3 3 19 1 7 7 13
Vector after sorting:
1 1 2 3 3 5 6 6 6 7 7 8 8 8 8 10 10 11 13 16 17 19 20 22 24

Array before sorting:
34 37 13 9 29| 0 25 41 7 42|
21 20 28 16 3| 3 32 17 45 40|
11 5 12 7 9| 11 41 33 1 17|
38 30 5 46 38| 38 23 40 1 43|
35 1 5 43 48| 14 10 38 8 0|

Array after sorting:
1 7 13 29 38| 0 7 17 33 41|
3 9 16 30 38| 0 8 17 38 41|
5 9 20 34 43| 1 10 23 38 42|
5 11 21 35 46| 1 11 25 40 43|
5 12 28 37 48| 3 14 32 40 45|
```

InsertExchange

1) Демонстрація результатів сортування для гібридного алгоритму "вставка – обмін" при заповненні впорядковано.

```
D:\University\sda\Kurswork\kurswork\bin\Debug\kurswork.exe
Demonstration of InsertExchange algorithm on Sorted Array and Vector
Size P:2 M:5 N:5
Vector before sorting:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
Vector after sorting:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
Array before sorting:
 0  5 10 15 20| 25 30 35 40 45|
 1  6 11 16 21| 26 31 36 41 46|
 2  7 12 17 22| 27 32 37 42 47|
 3  8 13 18 23| 28 33 38 43 48|
 4  9 14 19 24| 29 34 39 44 49|
Array after sorting:
 0  5 10 15 20| 25 30 35 40 45|
 1  6 11 16 21| 26 31 36 41 46|
 2  7 12 17 22| 27 32 37 42 47|
 3  8 13 18 23| 28 33 38 43 48|
 4  9 14 19 24| 29 34 39 44 49|
```

2) Демонстрація результатів сортування для гібридного алгоритму "вставка – обмін" при заповненні обернено-впорядковано.

```
D:\University\sda\Kurswork\kurswork\bin\Debug\kurswork.exe
Demonstration of InsertExchange algorithm on BackOrdered Array and Vector
Size P:2 M:5 N:5
Vector before sorting:
24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
Vector after sorting:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
Array before sorting:
50 45 40 35 30| 25 20 15 10 5|
49 44 39 34 29| 24 19 14 9 4|
48 43 38 33 28| 23 18 13 8 3|
47 42 37 32 27| 22 17 12 7 2|
46 41 36 31 26| 21 16 11 6 1|
Array after sorting:
26 31 36 41 46| 1 6 11 16 21|
27 32 37 42 47| 2 7 12 17 22|
28 33 38 43 48| 3 8 13 18 23|
29 34 39 44 49| 4 9 14 19 24|
30 35 40 45 50| 5 10 15 20 25|
```

3) Демонстрація результатів сортування для гібридного алгоритму "вставка – обмін" при заповненні випадково.

```
D:\University\sda\Kurswork\kurswork\bin\Debug\kurswork.exe
Demonstration of InsertExchange algorithm on Random Array and Vector
Size P:2 M:5 N:5

Vector before sorting:
9 12 0 13 12 2 21 4 24 20 11 24 17 12 20 1 16 10 3 16 19 24 13 21 14
Vector after sorting:
0 1 2 3 4 9 10 11 12 12 12 13 13 14 16 16 17 19 20 20 21 21 24 24 24

Array before sorting:
 1 21 15 42 3| 20 1 4 48 35|
10 37 35 3 42| 27 49 43 13 29|
33 12 27 12 40| 24 17 5 18 42|
41 35 1 49 37| 39 4 37 11 39|
44 14 32 32 34| 16 39 7 12 3|

Array after sorting:
 1 12 27 35 41| 1 7 17 29 39|
 1 12 32 35 42| 3 11 18 35 42|
 3 14 32 37 42| 4 12 20 37 43|
 3 15 33 37 44| 4 13 24 39 48|
10 21 34 40 49| 5 16 27 39 49|
```

Select3Exchange

1) Демонстрація результатів сортування для гібридного алгоритму "вибірN³ – обмін" при заповненні впорядковано.

```
D:\University\sda\Kurswork\kurswork\bin\Debug\kurswork.exe
Demonstration of Select3Exchange algorithm on Sorted Array and Vector
Size P:2 M:5 N:5

Vector before sorting:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
Vector after sorting:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24

Array before sorting:
 0 5 10 15 20| 25 30 35 40 45|
 1 6 11 16 21| 26 31 36 41 46|
 2 7 12 17 22| 27 32 37 42 47|
 3 8 13 18 23| 28 33 38 43 48|
 4 9 14 19 24| 29 34 39 44 49|

Array after sorting:
 0 5 10 15 20| 25 30 35 40 45|
 1 6 11 16 21| 26 31 36 41 46|
 2 7 12 17 22| 27 32 37 42 47|
 3 8 13 18 23| 28 33 38 43 48|
 4 9 14 19 24| 29 34 39 44 49|
```

2) Демонстрація результатів сортування для гібридного алгоритму "вибірN^o3 – обмін" при заповненні обернено-впорядковано.

```
D:\University\sda\Kurswork\kurswork\bin\Debug\kurswork.exe
Demonstration of Select3Exchange algorithm on BackOrdered Array and Vector

Size P:2 M:5 N:5

Vector before sorting:
24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
Vector after sorting:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24

Array before sorting:
50 45 40 35 30| 25 20 15 10 5|
49 44 39 34 29| 24 19 14 9 4|
48 43 38 33 28| 23 18 13 8 3|
47 42 37 32 27| 22 17 12 7 2|
46 41 36 31 26| 21 16 11 6 1|

Array after sorting:
26 31 36 41 46| 1 6 11 16 21|
27 32 37 42 47| 2 7 12 17 22|
28 33 38 43 48| 3 8 13 18 23|
29 34 39 44 49| 4 9 14 19 24|
30 35 40 45 50| 5 10 15 20 25|
```

3) Демонстрація результатів сортування для гібридного алгоритму "вставка – обмін" при заповненні випадково.

```
D:\University\sda\Kurswork\kurswork\bin\Debug\kurswork.exe
Demonstration of Select3Exchange algorithm on Random Array and Vector

Size P:2 M:5 N:5

Vector before sorting:
24 5 4 15 6 12 2 8 0 4 8 21 17 20 13 14 1 21 4 16 5 23 15 6 10
Vector after sorting:
0 1 2 4 4 4 5 5 6 6 8 8 10 12 13 14 15 15 16 17 20 21 21 23 24

Array before sorting:
31 36 2 49 40| 35 9 2 26 1|
22 3 7 1 24| 41 19 22 16 25|
11 25 32 5 0| 49 47 42 25 6|
31 4 34 42 32| 16 41 6 40 44|
5 48 15 39 32| 32 7 17 29 1|

Array after sorting:
0 5 22 32 39| 1 7 19 29 41|
1 5 24 32 40| 1 9 22 32 42|
2 7 25 32 42| 2 16 25 35 44|
3 11 31 34 48| 6 16 25 40 47|
4 15 31 36 49| 6 17 26 41 49|
```

Інформація про систему, на якій виконувалася курсова робота

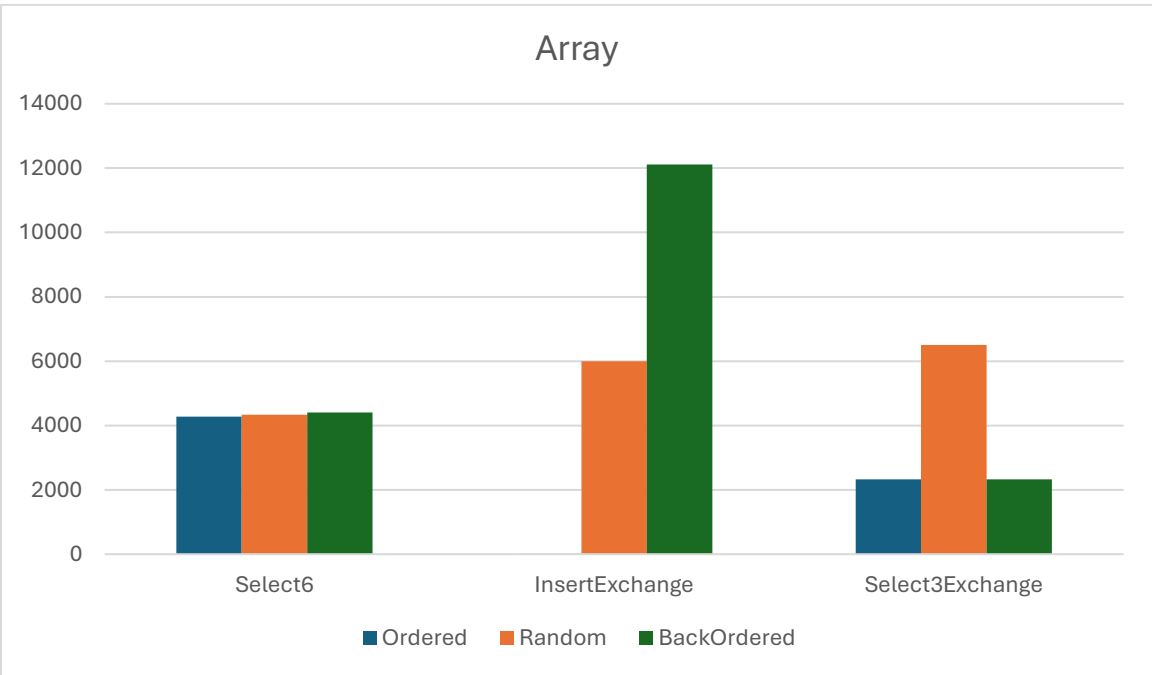
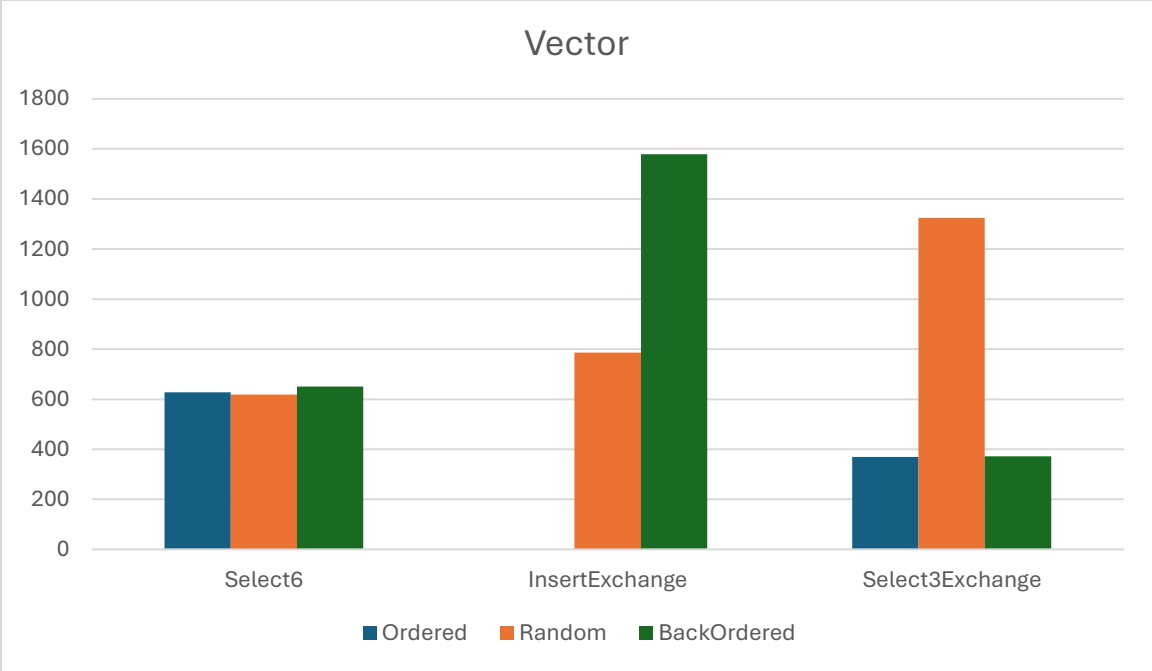
Item	Value
OS Name	Microsoft Windows 11 Pro N
Processor	12th Gen Intel(R) Core(TM) i3-1215U, 1200 Mhz, 6 Core(s), 8 Logical Proces...
Installed Physical Memory (RA...	16.0 GB

ВИПАДКИ ДОСЛІДЖЕННЯ

Випадок дослідження I. Залежність часу роботи алгоритмів від форми перерізу масиву.

P = const = 3, N = 20000, M = 2, VectorLength = 40000

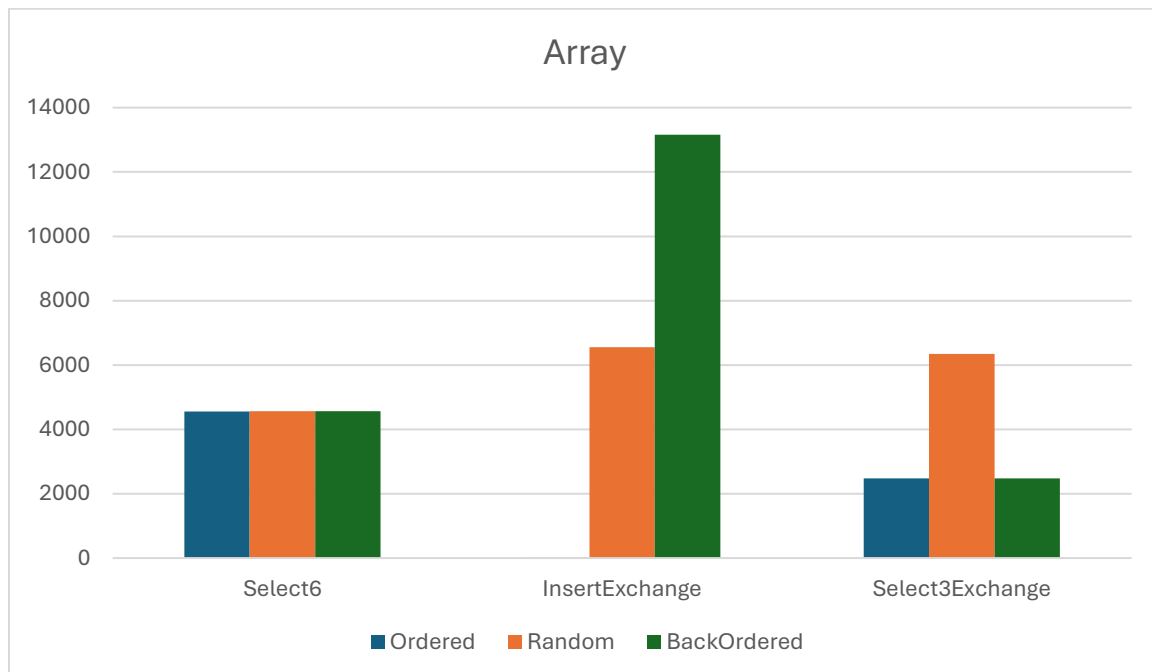
C:\University\sda\Kurswork\k			
Vector			
Size M*N:40000			
	Ordered	Random	BackOrdered
Select6	628.250	618.650	651.250
InsertExchange	0.00000000	786.500	1579.150
Select3Exchange	369.950	1324.050	372.000
Array			
Size P:3 M:2 N:20000			
	Ordered	Random	BackOrdered
Select6	4281.150	4332.700	4406.000
InsertExchange	0.00000000	6000.200	12110.650
Select3Exchange	2329.250	6506.550	2325.200



2. $P = \text{const} = 3$, $N = 2000$, $M = 20$, $\text{VectorLength} = 40000$

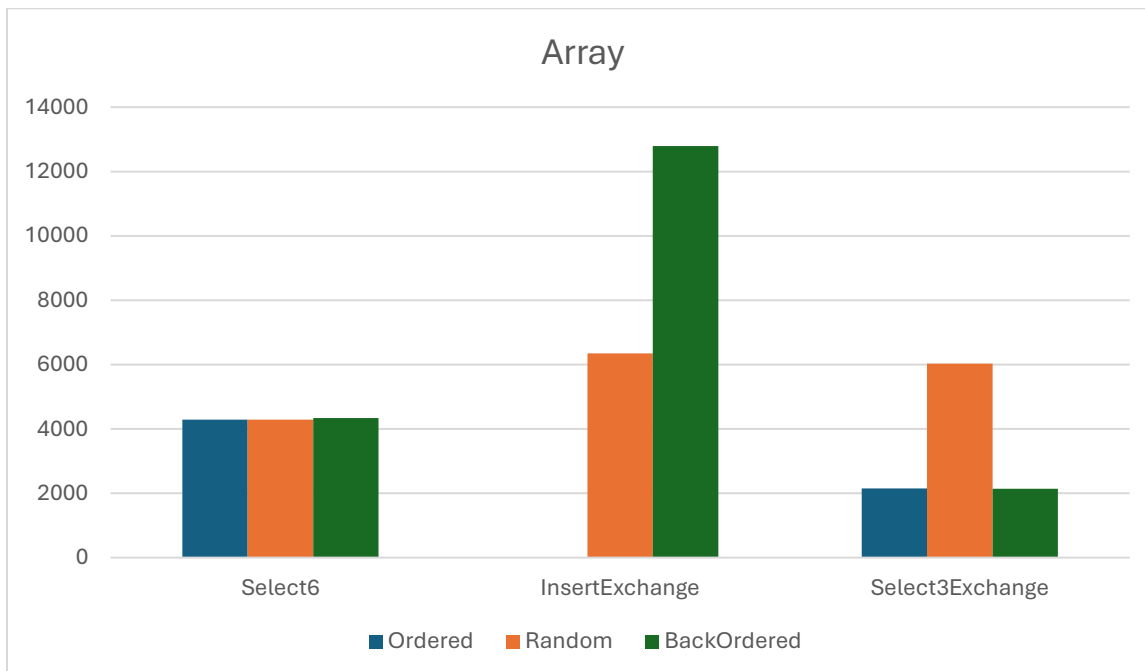
Vector			
Size M*N:40000			
	Ordered	Random	BackOrdered
Select6	632.100	624.150	638.800
InsertExchange	0.00000000	785.900	1571.350
Select3Exchange	365.050	1322.700	363.700

Array			
Size P:3 M:20 N:2000			
	Ordered	Random	BackOrdered
Select6	4553.800	4563.600	4562.150
InsertExchange	0.00000000	6558.250	13158.300
Select3Exchange	2480.500	6348.950	2477.050



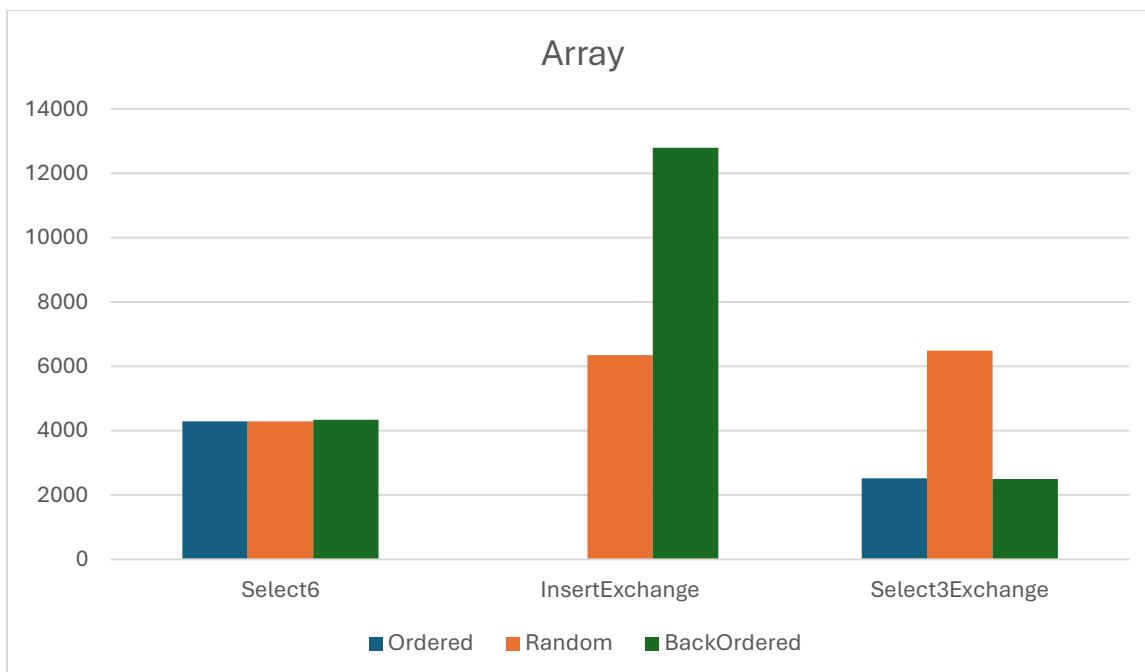
3. $P = \text{const} = 3$, $N = 200$, $M = 200$, $\text{VectorLength} = 40000$

Vector			
Size $M \times N: 40000$			
	Ordered	Random	BackOrdered
Select6	644.350	639.950	663.950
InsertExchange	0.00000000	773.900	1554.100
Select3Exchange	373.800	1275.700	369.850
Array			
Size $P:3 \ M:200 \ N:200$			
	Ordered	Random	BackOrdered
Select6	4285.050	4286.200	4339.050
InsertExchange	0.00000000	6346.050	12792.900
Select3Exchange	2149.600098	6027.049805	2141.449951



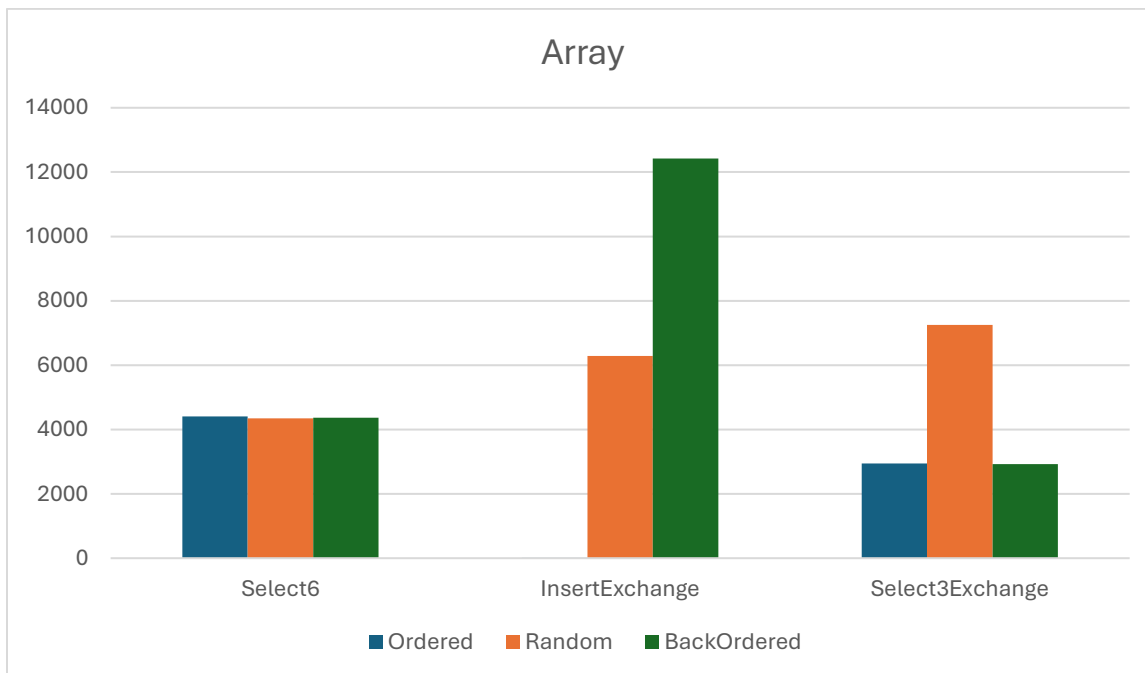
4. $P = \text{const} = 3$, $N = 20$, $M = 2000$, $\text{VectorLength} = 40000$

Vector			
Size $M \times N: 40000$			
	Ordered	Random	BackOrdered
Select6	638.450	639.000	641.650
InsertExchange	0.00000000	782.100	1562.300
Select3Exchange	373.750	1324.550	373.050
Array			
Size $P:3$ $M:2000$ $N:20$			
	Ordered	Random	BackOrdered
Select6	4739.150	4921.950	4737.300
InsertExchange	0.00000000	6763.750	13424.400
Select3Exchange	2518.600	6491.000	2495.250



5. $P = \text{const} = 3$, $N = 2$, $M = 20000$, $\text{VectorLength} = 40000$

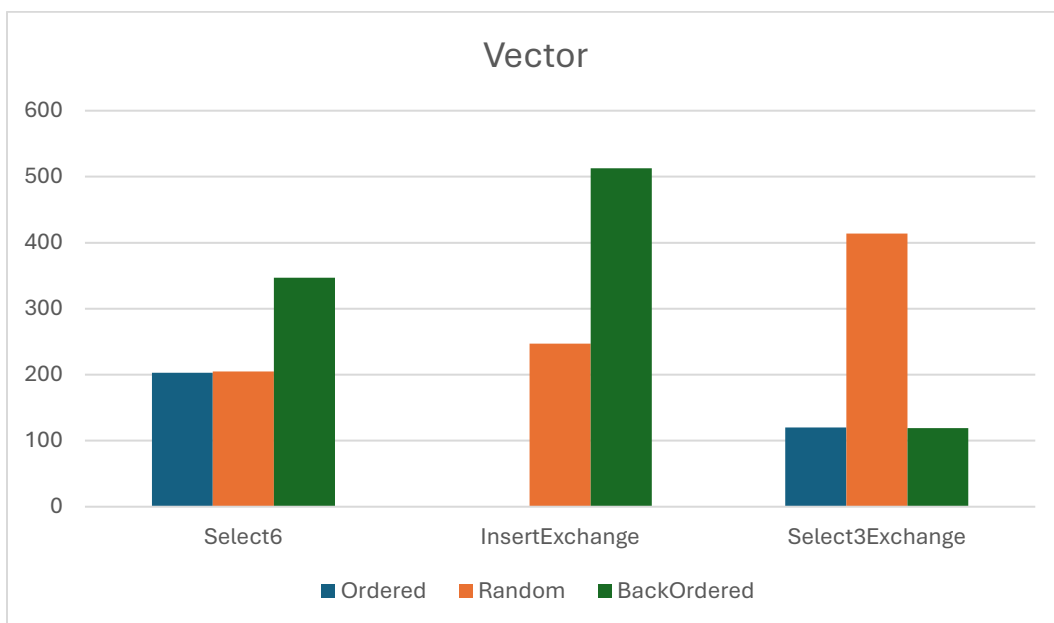
Vector			
Size M*N:40000			
	Ordered	Random	BackOrdered
Select6	628.350	631.150	662.450
InsertExchange	0.00000000	806.950	1556.950
Select3Exchange	369.950	1315.050	369.500
Array			
Size P:3 M:20000 N:2			
	Ordered	Random	BackOrdered
Select6	4405.700	4344.500	4364.300
InsertExchange	0.00000000	6289.050	12419.000
Select3Exchange	2941.650	7248.100	2927.550



Випадок дослідження II. Залежність часу роботи алгоритмів від кількості перерізів масиву

P = 2 , N = M = 150, VectorLength = 22500

C:\University\sda\Kurswork\k			
Vector			
Size M*N:22500			
	Ordered	Random	BackOrdered
Select6	203.000	205.500	347.350
InsertExchange	0.00000000	247.050	513.850
Select3Exchange	120.700	414.350	119.600
Array			
Size P:2 M:150 N:150			
	Ordered	Random	BackOrdered
Select6	874.650	890.400	901.400
InsertExchange	0.00000000	1333.150	2674.650
Select3Exchange	451.050	1412.200	453.050



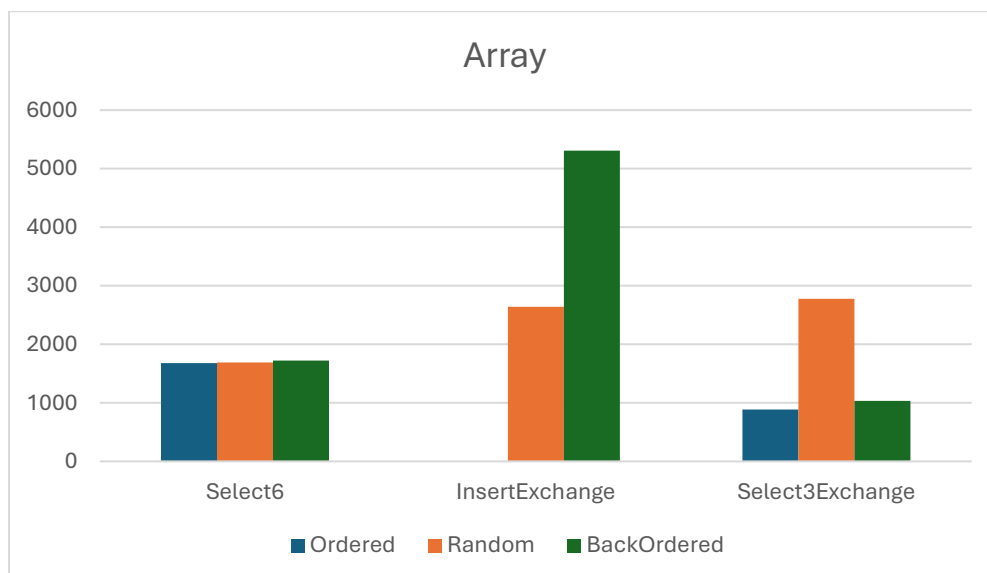


P = 4 , N = M = 150, VectorLength = 22500

C:\University\sda\Kurswork\k

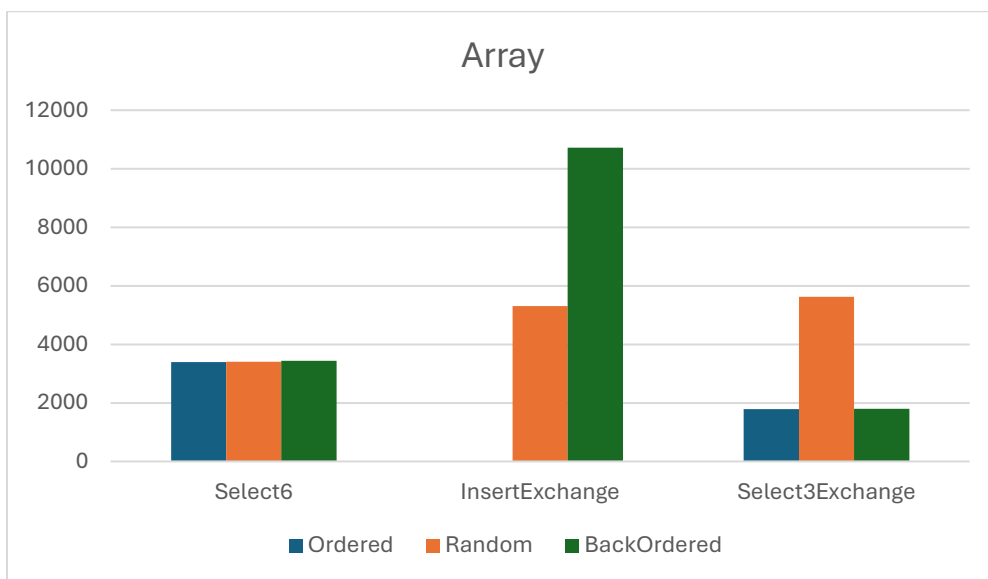
Array
Size P:4 M:150 N:150

	Ordered	Random	BackOrdered
Select6	1676.900	1687.750	1721.300
InsertExchange	0.00000000	2639.500	5310.000
Select3Exchange	888.350	2778.800	1034.500



P = 8 , N = M = 150, VectorLength = 22500

C:\University\sda\Kurswork\k			
Array			
Size P:8 M:150 N:150			
	Ordered	Random	BackOrdered
Select6	3396.250000	3409.449951	3437.350098
InsertExchange	0.000000	5310.549805	10715.700195
Select3Exchange	1784.099976	5625.549805	1802.699951



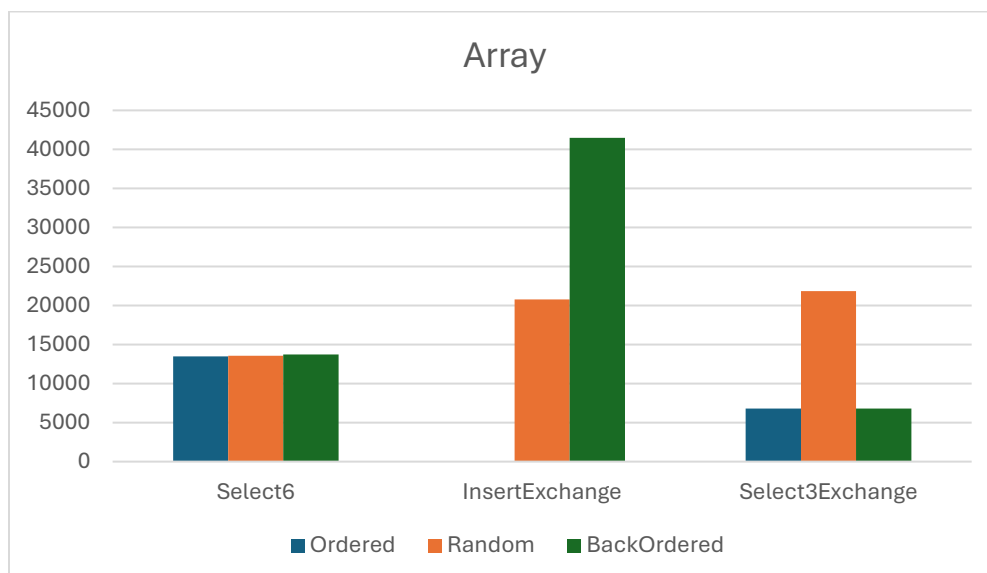
P = 16 , N = M = 150, VectorLength = 22500

C:\University\sda\Kurswork\k			
Array			
Size P:16 M:150 N:150			
	Ordered	Random	BackOrdered
Select6	10482.200	10537.800	10565.700
InsertExchange	0.00000000	15829.900	31695.850
Select3Exchange	5260.500	15863.700	5245.100



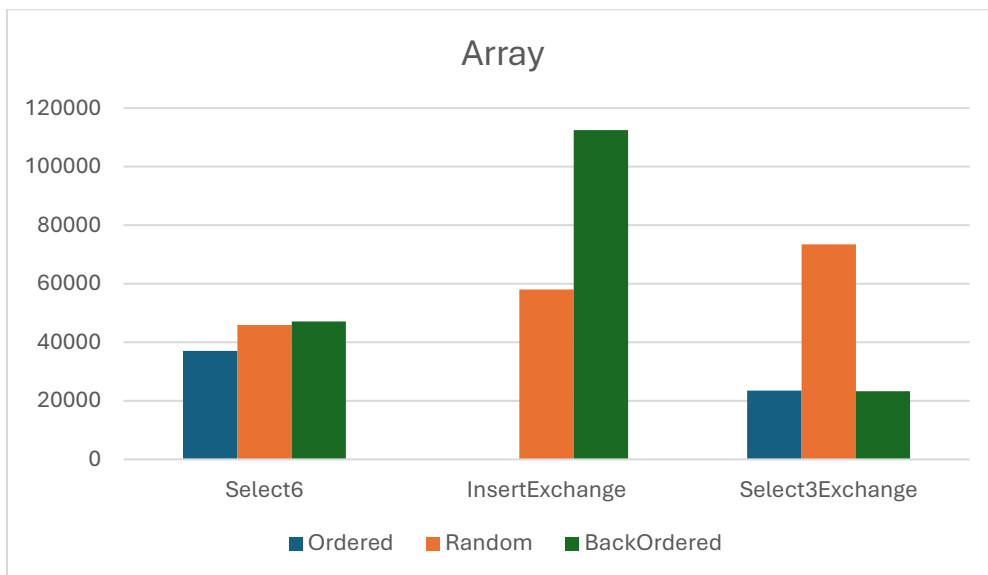
P = 32 , N = M = 150, VectorLength = 22500

C:\University\sda\Kurswork\k			
Array			
Size P:32 M:150 N:150			
	Ordered	Random	BackOrdered
Select6	13500.800	13556.350	13738.550
InsertExchange	0.00000000	20777.900	41493.750
Select3Exchange	6830.250000	21712.550781	6778.000000



P = 64 , N = M = 150, VectorLength = 22500

C:\University\sda\Kurswork\k			
Array			
Size P:64 M:150 N:150			
	Ordered	Random	BackOrdered
Select6	37025.102	45934.602	47140.102
InsertExchange	7.80000019	57991.051	112521.547
Select3Exchange	23504.350	73450.148	23302.400



Випадок дослідження III. Залежність часу роботи алгоритмів

від розміру перерізів масиву

P = 3, N = 4, M = 4

C:\University\sda\Kurswork\k			
Vector			
Size M*N:16			
	Ordered	Random	BackOrdered
Select6	0.000000	0.000000	0.000000
InsertExchange	0.000000	0.000000	0.000000
Select3Exchange	0.000000	0.000000	0.000000


```

Array
Size P:3 M:4 N:4

```

	Ordered	Random	BackOrdered
Select6	0.000000	0.000000	0.000000
InsertExchange	0.000000	0.000000	0.000000
Select3Exchange	0.000000	0.000000	0.000000

P = 3, N = 8, M = 8

```

C:\University\sda\Kurswork\k
Vector
Size M*N:64

```

	Ordered	Random	BackOrdered
Select6	0.000000	0.000000	0.000000
InsertExchange	0.000000	0.000000	0.000000
Select3Exchange	0.000000	0.000000	0.000000

```

Array
Size P:3 M:8 N:8

```

	Ordered	Random	BackOrdered
Select6	0.000000	0.000000	0.000000
InsertExchange	0.000000	0.000000	0.000000
Select3Exchange	0.000000	0.000000	0.000000

P = 3, N = 16, M = 16

```

C:\University\sda\Kurswork\k
Vector
Size M*N:256

```

	Ordered	Random	BackOrdered
Select6	0.000000	0.000000	0.000000
InsertExchange	0.000000	0.000000	0.000000
Select3Exchange	0.000000	0.000000	0.000000

```

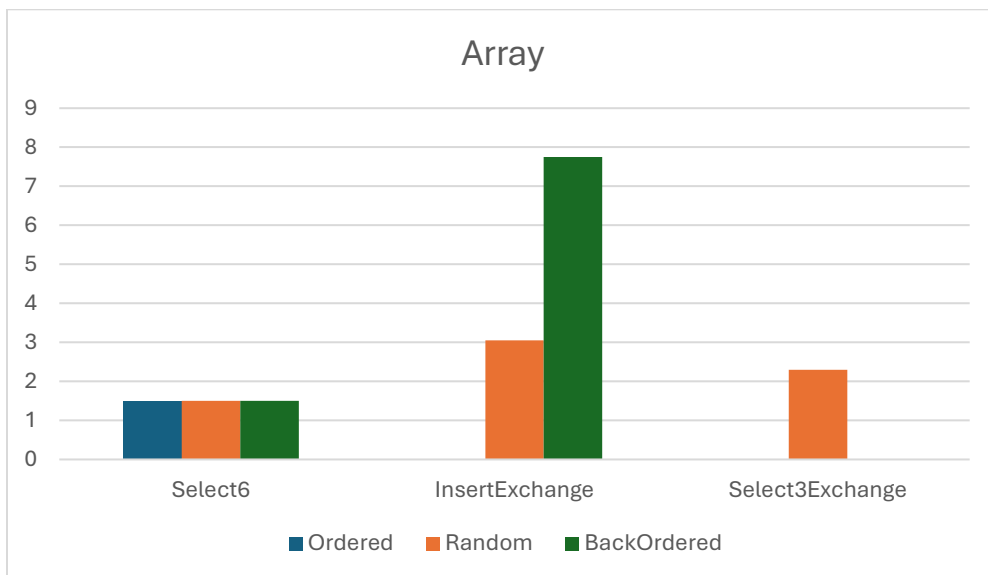
Array
Size P:3 M:16 N:16

```

	Ordered	Random	BackOrdered
Select6	0.000000	0.000000	0.000000
InsertExchange	0.000000	0.000000	0.000000
Select3Exchange	0.000000	0.000000	0.000000

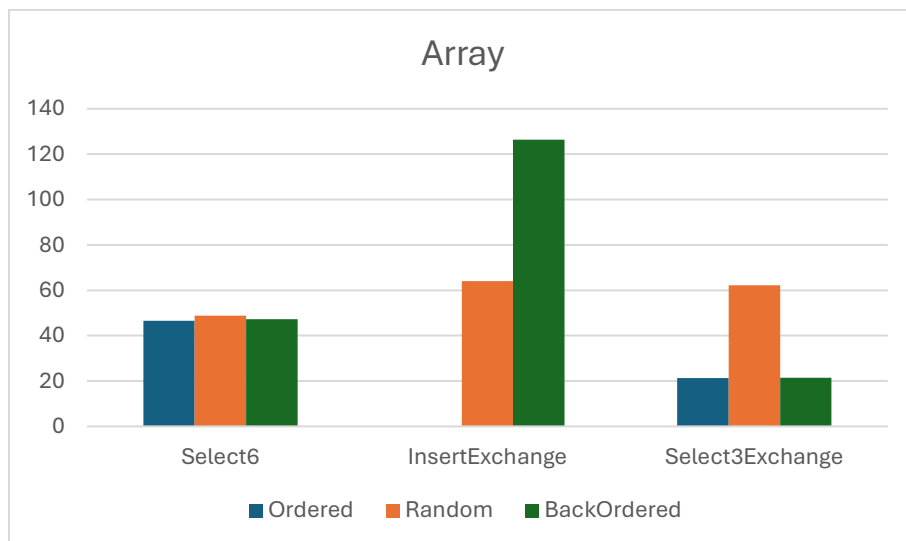
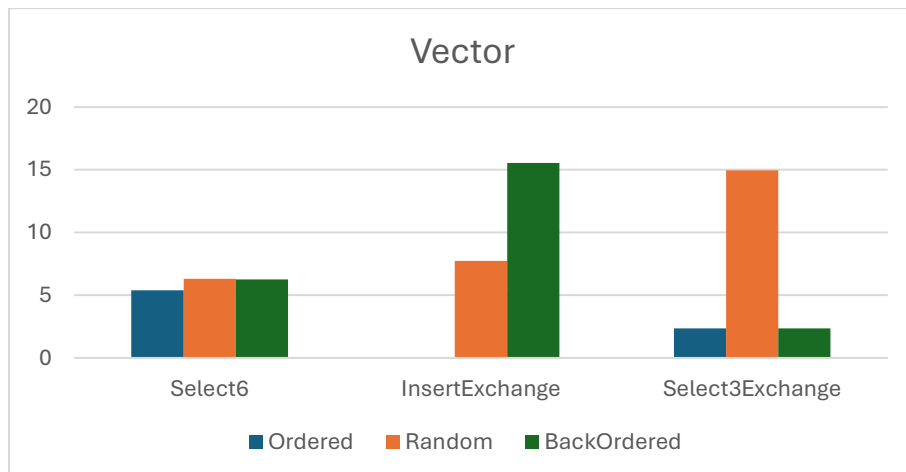
P = 3, N = 32, M = 32

C:\University\sda\Kurswork\k			
Vector			
Size M*N:1024			
	Ordered	Random	BackOrdered
Select6	0.000000	0.000000	0.000000
InsertExchange	0.000000	0.000000	0.000000
Select3Exchange	0.000000	0.000000	0.000000
Array			
Size P:3 M:32 N:32			
	Ordered	Random	BackOrdered
Select6	1.500000	1.500000	1.500000
InsertExchange	0.000000	3.050000	7.750000
Select3Exchange	0.000000	2.300000	0.000000



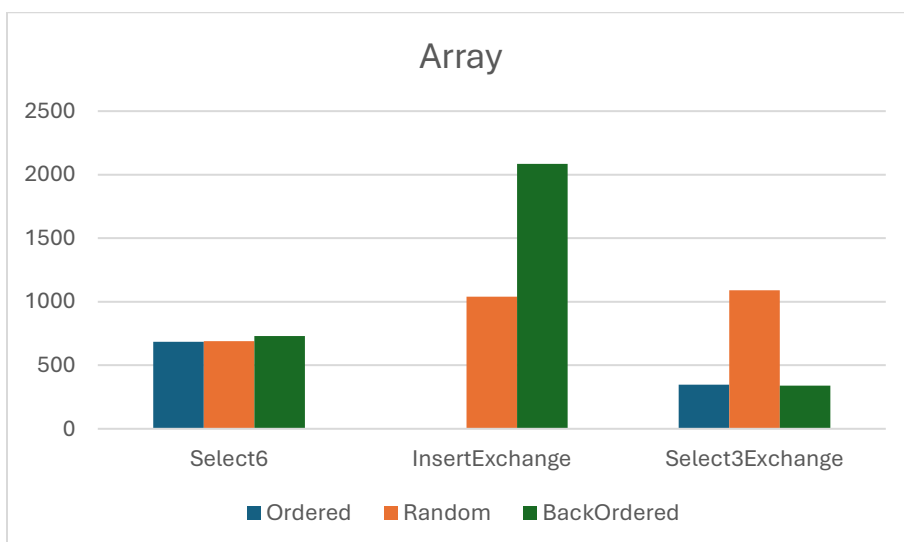
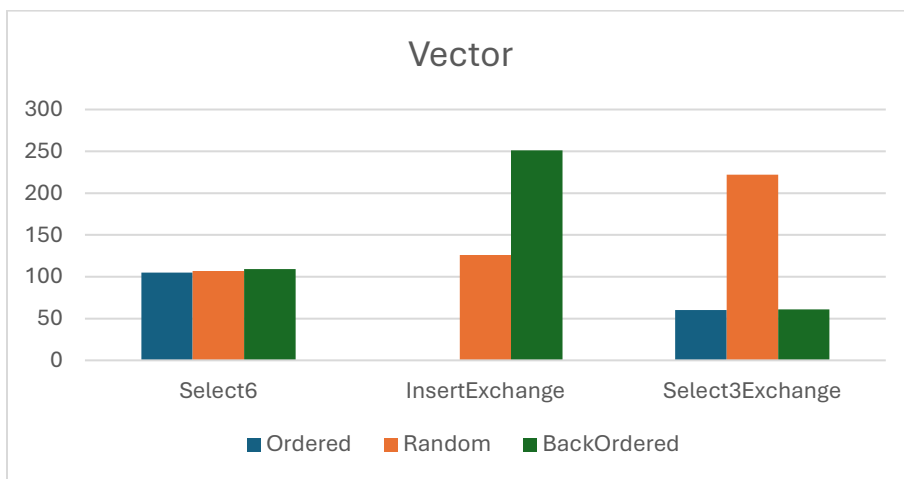
P = 3, N = 64, M = 64

C:\University\sda\Kurswork\k			
Vector			
Size M*N:4096			
	Ordered	Random	BackOrdered
Select6	5.400000	6.300000	6.250000
InsertExchange	0.000000	7.750000	15.550000
Select3Exchange	2.350000	14.950000	2.350000
Array			
Size P:3 M:64 N:64			
	Ordered	Random	BackOrdered
Select6	46.599998	48.750000	47.299999
InsertExchange	0.000000	64.050003	126.349998
Select3Exchange	21.250000	62.200001	21.400000



P = 3, N = 128, M = 128

C:\University\sda\Kurswork\k			
Vector			
Size M*N:16384			
	Ordered	Random	BackOrdered
Select6	105.550003	107.050003	109.250000
InsertExchange	0.000000	126.199997	251.850006
Select3Exchange	60.799999	222.000000	61.650002
Array			
Size P:3 M:128 N:128			
	Ordered	Random	BackOrdered
Select6	686.750000	691.299988	729.799988
InsertExchange	0.000000	1039.199951	2085.050049
Select3Exchange	348.049988	1091.199951	341.649994



P = 3, N = 256, M = 256

C:\University\sda\Kurswork\k

Vector

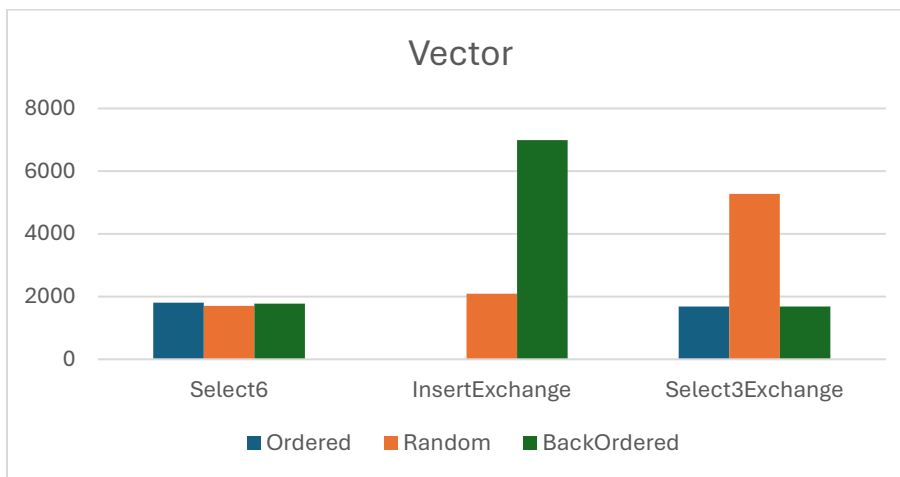
Size M*N:65536

	Ordered	Random	BackOrdered
Select6	1806.949951	1707.300049	1776.750000
InsertExchange	0.000000	2089.449951	6993.700195
Select3Exchange	1681.599976	5272.700195	1682.650024

Array

Size P:3 M:256 N:256

	Ordered	Random	BackOrdered
Select6	10801.000000	10786.799805	10960.400391
InsertExchange	0.000000	16567.400391	33209.398438
Select3Exchange	5372.100098	13983.849609	5394.000000



Порівняльний аналіз роботи алгоритмів на багатовимірних масивах, згідно запропонованих випадків дослідження

Спочатку розглянемо роботу кожного з алгоритмів окремо. Алгоритм сортування №6 методу прямого вибору працює на основі **for** циклів та умовних конструкцій. Через що, незалежно від способу заповнення масиву, ці проходи по циклу та перевірки будуть задіяні в кожному з випадків. Також важливо додати що середня кількість присвоєнь прямого методу вибору має порядок $R = O(n \cdot \ln(n))$. Кількість присвоєнь в цьому алгоритмі є дуже малою, оскільки він працює за рахунок пошуку мінімального елемента. Також, саме в цьому алгоритмі завдяки додатковій перевірці, переприсвоєння мінімуму на те саме місце не виконується. Враховуючи це все, алгоритм показує майже однаковий час для кожного з варіантів заповнення як масиву так і вектора.

Гібридний алгоритм "вставка – обмін" працює за рахунок циклу **for**, та вкладеного циклу **while**, який спрацьовує за певної умови. За рахунок цього, при початковому заповненні масиву і вектора впорядкованими значеннями, умова циклу **while** не виконується. Через що реалізовується звичайний лінійний прохід по циклу **for**. З цього впливає лінійна залежність **O(n)** і з цим величезна швидкість роботи алгоритму при такій умові початкового заповнення. При заповненні масиву обернено-впорядкованим варіантом, навпаки ж, цикл **while** буде спрацьовувати кожного разу. Обмінювати між собою кожен сусідній елемент, таким чином будучи найменш ефективним серед всіх варіантів. Оскільки при випадковому заповненні з великою вірогідністю якась кількість елементів вже буде між собою впорядкована, що дасть змогу менше разів заходити в цикл **while** та обмінювати між собою елементи. В такому випадку він відсортує швидше ніж при обернено-впорядкованому масиві чи векторі.

Гібридний алгоритм "вибір №3 – обмін" працює за допомогою **for** циклів та умовних конструкцій. При заповненні масиву чи вектора впорядковано, час витрачається на прохід по циклам та перевірку умовними конструкціями. Але жодна з умовних конструкцій не справджується, оскільки вони перевіряють чи подальші елементи більші

за початково встановлений мінімум чи максимум. При заповненні масиву чи вектора обернено-впорядковано лише перша перевірка спрацьовує та переставляє мінімум та максимум місцями, через що з отриманих результатів припускаю, що це не грає суттєвої ролі в часі. Тому ці два випадки можна вважати рівними в часі. У випадку заповнення масиву чи вектора випадковими значеннями, при заміні окремих елементів на мінімум чи максимум, буде витратись додатковий час. Тому сортування при такому початковому заповненні займе найбільшу кількість часу.

Перейдемо до першого випадку дослідження, залежність часу роботи алгоритмів від форми перерізу масиву. З результатів тестувань впливає що час роботи жодного з цих алгоритмів не залежить від форми перерізу масиву. На час роботи впливає тільки кількість елементів в цьому перерізі. Два з трьох алгоритмів використовуються в якості інструмента проходження по матриці чи вектору звичайні **for** цикли, що не дає жодного пришвидшення при проході по стовпцям чи рядкам. Гібридний алгоритм "вставка – обмін" який має в своїй основі умовний цикл **while** також з однаковою швидкістю крокує як по рядкам, так і по стовпцям. І це досить логічного, оскільки жоден з цих алгоритмів не зав'язаний на більш швидкій обробці стовпців чи рядків.

Випадок дослідження два. Залежність часу роботи алгоритмів від кількості перерізів масиву показав, що час сортування збільшується в стільки разів у скільки збільшується кількість перерізів. У нашому випадку це геометрична прогресія зі збільшенням в двоє. Це ще раз підтверджу правильність висновку про те що, час роботи цих алгоритмів залежить виключно від кількості елементів в масиві чи векторі, та не пов'язаний з формою перерізу.

Випадок дослідження три. Залежність часу роботи алгоритмів від розміру перерізів масиву демонструє що час в середньому зростає приблизно в 16 разів.

Висновок

В даній курсовій роботі мною було досліджено три алгоритми сортування:

Алгоритм №6 методу прямого вибору, Гібридний алгоритм "вставка – обмін", та Гібридний алгоритм "вибір№3 – обмін". Порівнявши швидкості їх роботи для різних випадків дослідження, я проаналізувала сильні та слабкі сторони кожного з цих алгоритмів. Визначившись, в яких випадках кожен з цих алгоритмів буде доцільно використовувати.

У випадку коли нам завчасно не відомо як буде початково заповнений масив чи вектор якій нам необхідно відсортувати, доцільно використати алгоритм №6 методу прямого вибору. Оскільки він з оптимальною швидкістю сортує кожен з випадків. Якщо ми завчасно знаємо що масив буде заповнений більш-менш впорядковано, неважливо обернено чи ні, доцільно використати гібридний алгоритм "вибір№3 – обмін", оскільки в середньому він показує чудовий час роботи для цих випадків. Гібридний алгоритм "вставка – обмін" абсолютно обходить по швидкості два інші алгоритми у випадку вже відсортованого заповнення. Але сильно програє у двох інших випадках.

Список використаної літератури

1. Вирт Н. Алгоритмы и структуры данных. М.: Мир, 1989.3.
2. Конспект лекцій з СДА
3. Марченко О. І. , Марченко О. О. Структури даних та алгоритми
Підручник у 2-х частинах. Частина 1, 2024